

Chapter 4 — Predicting Properties of Energetic Materials

Python Code Examples · Jupyter Notebook Edition

From data to models: building QSPR predictors

Block	Topic
0	Install, imports, shared dataset & helper functions
1	Multiple Linear Regression — simplest QSPR model
2	Partial Least Squares (PLS) — handles correlated descriptors
3	Support Vector Machine (SVM) — non-linear regression
4	Random Forest — ensemble, feature importance
5	Gradient Boosting (XGBoost / sklearn) — highest accuracy
6	Cross-validation — reliable performance estimation
7	Hyperparameter tuning — grid search and random search
8	Model comparison dashboard
9	Predict all properties — density, sensitivity, detonation velocity
10	Domain of applicability — know when NOT to trust the model

Run each cell in order — Shift+Enter.

Block 0 · Install, Imports, Shared Dataset & Helpers

Run once. Builds the clean feature matrix used by all later blocks.

```
In [3]: # Uncomment to install on first run
# !pip install rdkit mordred pandas matplotlib seaborn scikit-learn xgboost scipy

import re, math, warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
```

```

import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
from scipy import stats as spstats

# RDKit
from rdkit import Chem, DataStructs
from rdkit.Chem import Descriptors, rdMolDescriptors, MolToSmiles, MolFromSmiles
from rdkit.Chem import rdFingerprintGenerator

# Scikit-Learn
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.cross_decomposition import PLSRegression
from sklearn.svm import SVR
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.model_selection import (train_test_split, cross_val_score,
                                      KFold, GridSearchCV, RandomizedSearchCV)

from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import VarianceThreshold
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
from sklearn.pipeline import Pipeline

# XGBoost (pip install xgboost)
try:
    from xgboost import XGBRegressor
    XGB_OK = True
except ImportError:
    XGB_OK = False
    print('XGBoost not installed – Block 5 will use sklearn GradientBoosting instead.')
    print('Install: pip install xgboost')

from IPython.display import display, HTML
%matplotlib inline
plt.rcParams['figure.dpi'] = 120

```

— Shared 30-compound literature dataset —

#	Name	SMILES	Formula	MW	rho	IS	Td	Dv	Hf
['TNT',	'Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]',	'C7H5N3O6',	227.13,	1.654,	15.0,	300,	6900,	-54.4],	
['RDX',	'O=[N+]([O-])N1CN([N+](=O)[O-])CN([N+](=O)[O-])C1',	'C3H6N6O6',	222.12,	1.820,	7.4,	210,	8750,	86.3],	
['HMX',	'O=[N+]([O-])N1CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-])C1',	'C4H8N8O8',	296.16,	1.905,	7.4,	280,	9110,	75.0],	
['PETN',	'C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-])CO[N+](=O)[O-]',	'C5H8N4O12',	316.14,	1.778,	3.0,	160,	8400,	-538.5],	
['TATB',	'Nc1c([N+](=O)[O-])c(N)c([N+](=O)[O-])c(N)c1[N+](=O)[O-]',	'C6H6N6O6',	258.15,	1.939,	50.0,	360,	7350,	-139.7],	
['NT0',	'O=[N+]([O-])c1n[nH]c(=O)[nH]1',	'C2H2N4O3',	130.06,	1.930,	71.0,	273,	8000,	-91.7],	
['FOX-7',	'NC(=C([N+](=O)[O-])[N+](=O)[O-])N',	'C2H4N4O4',	148.08,	1.885,	25.0,	230,	8870,	-133.8],	
['DNAN',	'COc1ccc([N+](=O)[O-])cc1[N+](=O)[O-]',	'C7H6N2O5',	198.13,	1.321,	40.0,	190,	6900,	-182.7],	
['CL-20',	'O=[N+]([O-])N1CN2CN([N+](=O)[O-])CN1[N+](=O)[O-]',	'C6H6N12O12',	438.19,	2.044,	4.0,	230,	9400,	460.0],	
['Tetryl',	'CN([N+](=O)[O-])c1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]',	'C7H5N5O8',	287.14,	1.730,	3.0,	195,	7850,	18.9],	
['TNAZ',	'O=[N+]([O-])C1(CN([N+](=O)[O-])C1[N+](=O)[O-])[N+](=O)[O-]',	'C3H4N4O8',	240.08,	1.840,	12.0,	200,	8740,	17.0],	
['LLM-105',	'Nc1nc2c([N+](=O)[O-])n[nH]c2c([N+](=O)[O-])n1',	'C4H4N8O4',	228.12,	1.913,	40.0,	328,	8560,	135.6],	
['ANTA',	'Nc1nnc([N+](=O)[O-])[nH]1',	'C2H3N5O2',	129.08,	1.840,	60.0,	260,	8200,	28.0],	

```

['NQ',      'NC(=N)N[N+](=O)[O-]',      'CH4N4O2',    104.07, 1.710, 95.0, 245, 7900, -92.5],
['Picric',  'Oc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]', 'C6H3N3O7',   229.10, 1.763, 7.4, 300, 7350, -217.8],
['DATB',    'Nc1c([N+](=O)[O-])c(N)c([N+](=O)[O-])cc1[N+](=O)[O-]', 'C6H5N5O6',   243.13, 1.840, 50.0, 320, 7520, -143.5],
['TETNB',   'O=[N+](O- )c1c([N+](=O)[O-])c([N+](=O)[O-])cc([N+](=O)[O-])c1', 'C6H2N4O8',   258.10, 1.814, 8.0, 180, 7900, 125.0],
['HNS',     'O=[N+](O- )c1ccc(C=Cc2ccc([N+](=O)[O-])cc2[N+](=O)[O-])c([N+](=O)[O-])c1', 'C14H6N6O12', 450.22, 1.745, 7.0, 318, 7600, 22.5],
['NG',      'O[C@@H](CO[N+](=O)[O-])CO[N+](=O)[O-]',      'C3H5N3O9',   227.09, 1.591, 0.2, 200, 7700, -370.9],
['EDNA',    'O=[N+](O- )NCC[N+](=O)[O-]',      'C2H5N3O4',   135.08, 1.713, 5.0, 175, 7570, -148.0],
['BTNEU',   'O=[N+](O- )c1c([N+](=O)[O-])c([N+](=O)[O-])c([N+](=O)[O-])c([N+](=O)[O-])c1[N+](=O)[O-]', 'C6N6O12', 348.10, 1.980, 8.0, 230, 9000, 385.0],
['MHN',     'OC(CO[N+](=O)[O-])CO[N+](=O)[O-]',      'C3H6N2O7',   182.09, 1.450, 60.0, 175, 6500, -403.0],
['NONA',    'O=[N+](O- )c1c([N+](=O)[O-])c([N+](=O)[O-])c([N+](=O)[O-])c([N+](=O)[O-])c1[N+](=O)[O-]', 'C6HN5O10', 287.10, 1.870, 6.0, 220, 8700, 300.0],
['Heptyl',  'CC(C)(C)c1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]',      'C10H11N3O6', 269.21, 1.490, 20.0, 190, 6800, -136.0],
['DNAZ',    'O=[N+](O- )N1CC([N+](=O)[O-])C1',      'C3H5N3O4',   147.09, 1.699, 14.0, 155, 7550, -148.0],
['BTF',     'O=[N+](O- )c1c([N+](=O)[O-])c([N+](=O)[O-])c([N+](=O)[O-])c([N+](=O)[O-])n1[N+](=O)[O-]', 'C5N6O12', 334.10, 1.920, 5.0, 210, 8900, 420.0],
['AN',      'N.[N+](=O)(O- )O-]',      'H4N2O3',      80.04, 1.720, 100.0, 230, 4500, -365.6],
['AP',      '[NH4+].[O-]Cl(=O)(=O)=O',      'ClH4NO4',    117.49, 1.950, 110.0, 300, 5500, -295.8],
['Nitroguanidine', 'NC(=N)N[N+](=O)[O-]',      'CH4N4O2',    104.07, 1.770, 95.0, 250, 7900, -92.5],
['DATB-2',  'Nc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]',      'C6H5N4O6',   229.12, 1.797, 25.0, 295, 7480, -120.0],
]

```

```

In [4]: COLS = ['Name', 'SMILES', 'Formula', 'MW', 'Density', 'IS_J', 'Td_C', 'Dv_ms', 'Hf_kJmol']
df_raw = pd.DataFrame(RAW, columns=COLS)

```

```

def parse_formula(formula):
    counts = {}
    for elem in ['C', 'H', 'N', 'O']:
        m = re.search(rf'{elem}(\d*)', str(formula))
        counts[elem] = int(m.group(1)) if m else (1 if m else 0)
    return counts

```

```

fpngen = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=256)

```

```

def get_descriptors(row):
    mol = MolFromSmiles(row['SMILES'])
    if mol is None: return None
    e = parse_formula(row['Formula'])
    mw = float(row['MW'])
    ob = (1600/mw)*(2*e['C']+e['H']/2-e['O'])
    fp = fpngen.GetFingerprint(mol)
    arr = np.zeros(256, dtype=np.uint8)
    DataStructs.ConvertToNumpyArray(fp, arr)
    d = {
        'MW': mw, 'nN': e['N'], 'nO': e['O'], 'nC': e['C'],
        'nNO2': row['SMILES'].count('[N+](=O)[O-]'),
        'NC': round(e['N']/(e['C']+1e-9), 3), 'OB': round(ob, 2),
        'TPSA': round(Descriptors.TPSA(mol), 2),
        'LogP': round(Descriptors.MolLogP(mol), 3),
        'HBD': rdMolDescriptors.CalcNumHBD(mol),
        'HBA': rdMolDescriptors.CalcNumHBA(mol),
        'Rings': rdMolDescriptors.CalcNumRings(mol),
        'ArR': rdMolDescriptors.CalcNumAromaticRings(mol),
        'RotB': rdMolDescriptors.CalcNumRotatableBonds(mol),
    }

```

```

        'FrCSP3': round(Descriptors.FractionCSP3(mol), 4),
        'Chi0': round(Descriptors.Chi0(mol), 4),
        'Kappa1': round(Descriptors.Kappa1(mol), 4),
        'BalabanJ': round(Descriptors.BalabanJ(mol), 4),
    }
    for i, b in enumerate(arr): d[f'FP{i}'] = int(b)
    return d

# --- 1. Generate Descriptors & Track Valid Rows ---
desc_list = [get_descriptors(row) for _, row in df_raw.iterrows()]

# Create the mask to keep only rows where get_descriptors worked
valid_rows_mask = [bool(d) for d in desc_list]

# Align both DataFrames to the exact same 28 valid rows right here!
df_desc = pd.DataFrame([d for d in desc_list if d])
df_meta = df_raw.loc[valid_rows_mask, [c for c in COLS if c not in ['SMILES', 'Formula']]].reset_index(drop=True)

# --- 2. Feature Selection & Preprocessing (Now running on aligned 28 rows) ---
vt = VarianceThreshold(0.01)
X_vt = vt.fit_transform(df_desc)
feat_names = df_desc.columns[vt.get_support()].tolist()
X_df = pd.DataFrame(X_vt, columns=feat_names)

corr_mat = X_df.corr().abs()
upper = corr_mat.where(np.triu(np.ones(corr_mat.shape), k=1).astype(bool))
to_drop = [c for c in upper.columns if any(upper[c] > 0.92)]
X_clean = X_df.drop(columns=to_drop)

# --- 3. Scaling (Will correctly output shape: [28, num_features]) ---
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_clean)
feat_names_final = X_clean.columns.tolist()

# --- 4. Pull Out Aligned Target Arrays ---
y_IS = df_meta['IS_J'].astype(float).values
y_rho = df_meta['Density'].astype(float).values
y_Dv = df_meta['Dv_ms'].astype(float).values
y_Td = df_meta['Td_C'].astype(float).values
y_Hf = df_meta['Hf_kJmol'].astype(float).values
names = df_meta['Name'].values

# --- 5. Verify and Split ---
print(f"X_scaled shape: {X_scaled.shape}") # Expected: (28, n)
print(f"y_IS shape: {y_IS.shape}") # Expected: (28,)
print(f"names shape: {names.shape}") # Expected: (28,)

# This will now execute perfectly without errors!
X_tr, X_te, y_tr, y_te, n_tr, n_te = train_test_split(
    X_scaled, y_IS, names, test_size=0.30, random_state=42
)

```

```
def evaluate(name, model, Xtr, Xte, ytr, yte):
    model.fit(Xtr, ytr)
    p_tr = model.predict(Xtr)
    p_te = model.predict(Xte)
    return {
        'Model': name,
        'R2_train': round(r2_score(ytr, p_tr), 3),
        'R2_test': round(r2_score(yte, p_te), 3),
        'MAE_test': round(mean_absolute_error(yte, p_te), 2),
        'RMSE_test': round(np.sqrt(mean_squared_error(yte, p_te)), 2),
    }, model.predict(Xte)
```

```
print("B0 ✓ dataset ready:", X_scaled.shape)
```

```
X_scaled shape: (28, 83)
```

```
y_IS shape: (28,)
```

```
names shape: (28,)
```

```
B0 ✓ dataset ready: (28, 83)
```

```
[10:38:58] SMILES Parse Error: unclosed ring for input: 'O=[N+](O-)N1CN2CN([N+](=O)[O-])CN1[N+](=O)[O-]'
```

```
[10:38:58] Can't kekulize mol. Unkekulized atoms: 3 4 8 12 16
```

```
In [8]: COLS = ['Name', 'SMILES', 'Formula', 'MW', 'Density', 'IS_J', 'Td_C', 'Dv_ms', 'Hf_kJmol']
df_raw = pd.DataFrame(RAW, columns=COLS)
```

```
def parse_formula(formula):
    counts = {}
    for elem in ['C', 'H', 'N', 'O']:
        m = re.search(rf'{elem}(\d*)', str(formula))
        counts[elem] = int(m.group(1)) if m and m.group(1) else (1 if m else 0)
    return counts
```

```
fpngen = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=256)
```

```
def get_descriptors(row):
    mol = MolFromSmiles(row['SMILES'])
    if mol is None: return None
    e = parse_formula(row['Formula'])
    mw = float(row['MW'])
    ob = (1600/mw)*(2*e['C']+e['H']/2-e['O'])
    fp = fpngen.GetFingerprint(mol)
    arr = np.zeros(256, dtype=np.uint8)
    DataStructs.ConvertToNumpyArray(fp, arr)
    d = {
        'MW': mw,
        'nN': e['N'], 'nO': e['O'], 'nC': e['C'],
        'nNO2': row['SMILES'].count('[N+](=O)[O-]'),
        'NC': round(e['N']/(e['C']+1e-9), 3),
        'OB': round(ob, 2),
        'TPSA': round(Descriptors.TPSA(mol), 2),
        'LogP': round(Descriptors.MolLogP(mol), 3),
```

```

'HBD':    rdMolDescriptors.CalcNumHBD(mol),
'HBA':    rdMolDescriptors.CalcNumHBA(mol),
'Rings':  rdMolDescriptors.CalcNumRings(mol),
'ArR':    rdMolDescriptors.CalcNumAromaticRings(mol),
'RotB':   rdMolDescriptors.CalcNumRotatableBonds(mol),
'FrCSP3': round(Descriptors.FractionCSP3(mol), 4),
'Chi0':   round(Descriptors.Chi0(mol), 4),
'Kappa1': round(Descriptors.Kappa1(mol), 4),
'BalabanJ':round(Descriptors.BalabanJ(mol), 4),
}
for i, b in enumerate(arr): d[f'FP{i}'] = int(b)
return d

```

```

desc_list = [get_descriptors(row) for _, row in df_raw.iterrows()]
# Create the mask to keep only rows where get_descriptors worked
valid_rows_mask = [bool(d) for d in desc_list]

```

```

# Align both DataFrames to the exact same 28 valid rows right here!
df_desc = pd.DataFrame([d for d in desc_list if d])
df_meta = df_raw.loc[valid_rows_mask, [c for c in COLS if c not in ['SMILES', 'Formula']]].reset_index(drop=True)
#df_meta = df_raw[[c for c in COLS if c != 'SMILES' and c != 'Formula']].reset_index(drop=True)

```

```

# Remove near-zero variance
from sklearn.feature_selection import VarianceThreshold
vt = VarianceThreshold(0.01)
X_vt = vt.fit_transform(df_desc)
feat_names = df_desc.columns[vt.get_support()].tolist()
X_df = pd.DataFrame(X_vt, columns=feat_names)

```

```

# Remove highly correlated (>0.92)
corr_mat = X_df.corr().abs()
upper = corr_mat.where(np.triu(np.ones(corr_mat.shape), k=1).astype(bool))
to_drop = [c for c in upper.columns if any(upper[c] > 0.92)]
X_clean = X_df.drop(columns=to_drop)

```

```

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_clean)
feat_names_final = X_clean.columns.tolist()

```

```

# Targets
y_IS = df_meta['IS_J'].astype(float).values # impact sensitivity
y_rho = df_meta['Density'].astype(float).values # crystal density
y_Dv = df_meta['Dv_ms'].astype(float).values # detonation velocity
y_Td = df_meta['Td_C'].astype(float).values # decomp temperature
y_Hf = df_meta['Hf_kJmol'].astype(float).values # heat of formation
names = df_meta['Name'].values

```

```

# Shared 70/30 split
X_tr, X_te, y_tr, y_te, n_tr, n_te = train_test_split(

```

```

X_scaled, y_IS, names, test_size=0.30, random_state=42)

print(f'Dataset: {len(names)} compounds, {X_scaled.shape[1]} features')
print(f'Train: {len(X_tr)}, Test: {len(X_te)}')
print(f'Targets available: IS_J, Density, Dv_ms, Td_C, Hf_kJmol')

# — Shared evaluation helper —————
def evaluate(name, model, Xtr, Xte, ytr, yte):
    model.fit(Xtr, ytr)
    p_tr = model.predict(Xtr)
    p_te = model.predict(Xte)
    return {
        'Model': name,
        'R2_train': round(r2_score(ytr, p_tr), 3),
        'R2_test': round(r2_score(yte, p_te), 3),
        'MAE_test': round(mean_absolute_error(yte, p_te), 2),
        'RMSE_test': round(np.sqrt(mean_squared_error(yte, p_te)), 2),
    }, model.predict(Xte)

def obs_vs_pred(ax, y_obs, y_pred, title, unit=''):
    ax.scatter(y_obs, y_pred, color='steelblue', s=55, alpha=0.8, edgecolors='white')
    lo, hi = min(y_obs.min(), y_pred.min())*0.9, max(y_obs.max(), y_pred.max())*1.05
    ax.plot([lo, hi], [lo, hi], 'k--', lw=1)
    r2 = r2_score(y_obs, y_pred)
    ax.set_title(f'{title}\nR²={r2:.3f}', fontsize=9)
    ax.set_xlabel(f'Observed {unit}', fontsize=8)
    ax.set_ylabel(f'Predicted {unit}', fontsize=8)

print('✓ Setup complete')
print("B0 ✓ dataset ready:", X_scaled.shape)

```

Dataset: 28 compounds, 83 features

Train: 19, Test: 9

Targets available: IS_J, Density, Dv_ms, Td_C, Hf_kJmol

✓ Setup complete

B0 ✓ dataset ready: (28, 83)

[10:44:37] SMILES Parse Error: unclosed ring for input: 'O=[N+](O-)]N1CN2CN([N+](=O)[O-])CN1[N+](=O)[O-]'

[10:44:37] Can't kekulize mol. Unkekulized atoms: 3 4 8 12 16

Block 1 · Multiple Linear Regression (MLR)

The simplest QSPR model: a **weighted sum of descriptors**.

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

Variant	When to use
Plain MLR	Small datasets, few descriptors, purely linear relationships
Ridge (L2)	Many correlated descriptors — shrinks all weights
Lasso (L1)	Want automatic feature selection — sets unimportant weights to 0

In [13]: *# — 1a: Plain MLR, Ridge, Lasso – fit and compare —*

```
struct_feats = ['MW', 'nN', 'nO', 'nC', 'nNO2', 'NC', 'OB', 'TPSA', 'LogP', 'HBD', 'HBA', 'Rings', 'ArR', 'RotB', 'FrCSP3', 'Chi0', 'Kappa1']
struct_feats = [f for f in struct_feats if f in X_clean.columns]
X_struct = scaler.fit_transform(X_clean[struct_feats])
Xs_tr, Xs_te, ys_tr, ys_te = train_test_split(X_struct, y_IS, test_size=0.30, random_state=42)

for name, model in [('MLR', LinearRegression()),
                    ('Ridge', Ridge(alpha=1.0, solver='svd')),
                    ('Lasso', Lasso(alpha=0.5, max_iter=5000))]:
    res, _ = evaluate(name, model, Xs_tr, Xs_te, ys_tr, ys_te)
    print(f"B1 {res['Model']}: R2_test={res['R2_test']}")
```

B1 MLR: R2_test=-29.8

B1 Ridge: R2_test=0.711

B1 Lasso: R2_test=0.624

In [22]:

```
results_mlr = []
preds_mlr = {}
for name, model in [('MLR (plain)', LinearRegression()),
                    ('Ridge (α=1)', Ridge(alpha=1.0, solver='svd')),
                    ('Lasso (α=0.5)', Lasso(alpha=0.5, max_iter=5000))]:
    res, pred = evaluate(name, model, Xs_tr, Xs_te, ys_tr, ys_te)
    results_mlr.append(res)
    preds_mlr[name] = pred

df_mlr = pd.DataFrame(results_mlr)
display(df_mlr)
```

	Model	R2_train	R2_test	MAE_test	RMSE_test
0	MLR (plain)	0.886	-29.800	148.83	219.48
1	Ridge (α=1)	0.771	0.711	15.33	21.27
2	Lasso (α=0.5)	0.755	0.624	15.84	24.23

In [23]: *# — 1b: MLR coefficients – which descriptors matter most? —*

```
mlr_model = LinearRegression().fit(Xs_tr, ys_tr)
coef_df = pd.DataFrame({'Feature': struct_feats,
```



```

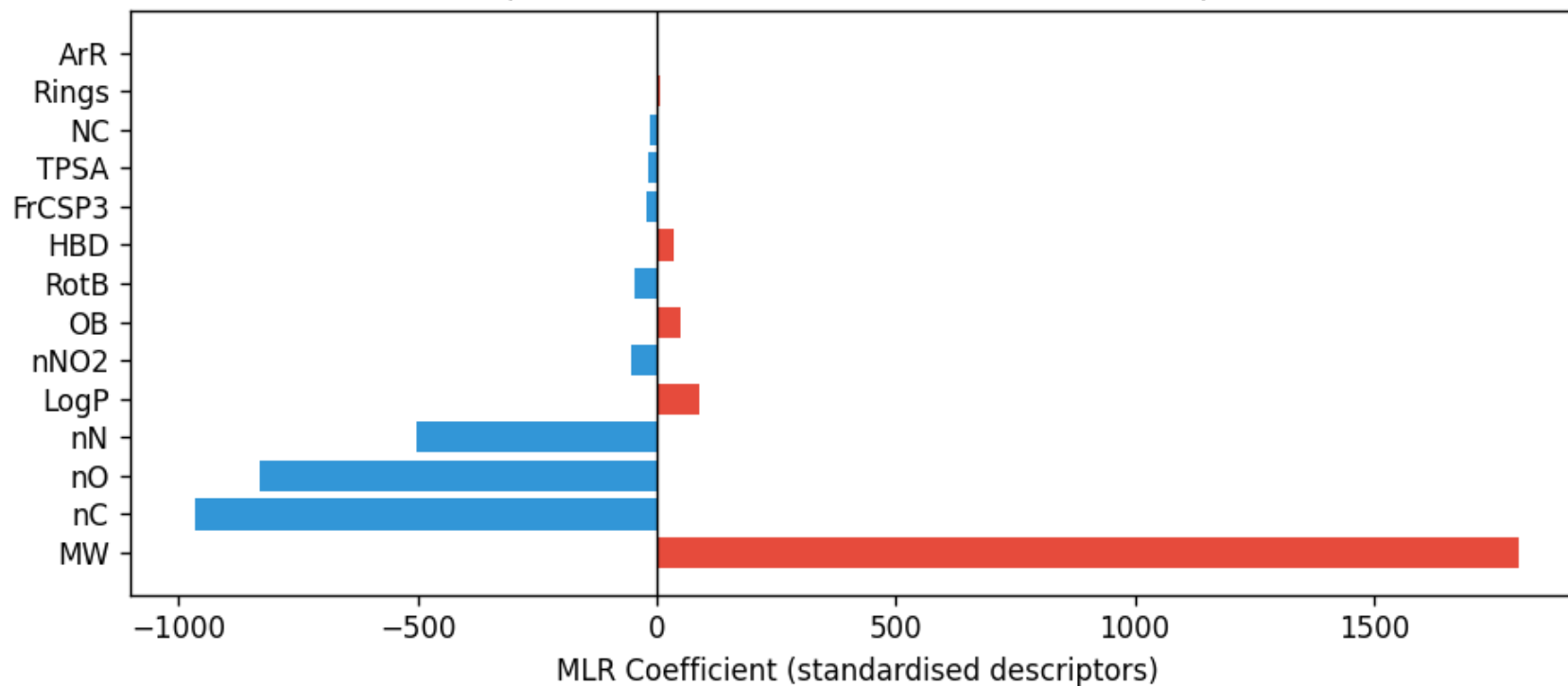
        'Coefficient': mlr_model.coef_))\
        .sort_values('Coefficient', key=abs, ascending=False)

fig, ax = plt.subplots(figsize=(8, 4))
colors = ['#e74c3c' if c > 0 else '#3498db' for c in coef_df['Coefficient']]
ax.barh(coef_df['Feature'], coef_df['Coefficient'], color=colors)
ax.axvline(0, color='black', linewidth=0.8)
ax.set_xlabel('MLR Coefficient (standardised descriptors)')
ax.set_title('MLR Coefficients for Impact Sensitivity Prediction\n'
            '(red = increases IS, blue = decreases IS)')
plt.tight_layout()
plt.show()

print('Intercept:', round(mlr_model.intercept_, 2))
print('\nTop 3 positive (increase sensitivity):',
      coef_df[coef_df['Coefficient']>0].head(3)['Feature'].tolist())
print('Top 3 negative (decrease sensitivity):',
      coef_df[coef_df['Coefficient']<0].head(3)['Feature'].tolist())

```

MLR Coefficients for Impact Sensitivity Prediction
(red = increases IS, blue = decreases IS)



Intercept: 44.2

Top 3 positive (increase sensitivity): ['MW', 'LogP', 'OB']

Top 3 negative (decrease sensitivity): ['nC', 'nO', 'nN']

```
In [24]: # — 1c: Lasso automatic feature selection —————  
  
from sklearn.linear_model import LassoCV  
  
lasso_cv = LassoCV(cv=5, max_iter=10000, random_state=42)  
lasso_cv.fit(Xs_tr, ys_tr)  
  
non_zero = [(feat, round(coef, 3))  
             for feat, coef in zip(struct_feats, lasso_cv.coef_) if coef != 0]  
zeroed    = [feat for feat, coef in zip(struct_feats, lasso_cv.coef_) if coef == 0]  
  
print(f'Best alpha (LassoCV): {lasso_cv.alpha_:.4f}')  
print(f'Non-zero features kept ({len(non_zero)}): {[f for f, _ in non_zero]}')  
print(f'Zeroed out ({len(zeroed)}): {zeroed}')  
r2_lassocv = r2_score(ys_te, lasso_cv.predict(Xs_te))  
print(f'LassoCV test R²: {r2_lassocv:.3f}')
```

Best alpha (LassoCV): 1.9693

Non-zero features kept (5): ['nN', 'nNO2', 'nC', 'HBD', 'FrCSP3']

Zeroed out (9): ['MW', 'nO', 'nC', 'OB', 'TPSA', 'LogP', 'Rings', 'ArR', 'RotB']

LassoCV test R²: 0.565

Block 2 · Partial Least Squares (PLS)

PLS is designed for datasets where **descriptors far outnumber compounds** and many descriptors are correlated.

It projects both X and y into a smaller set of **latent variables** that maximise covariance.

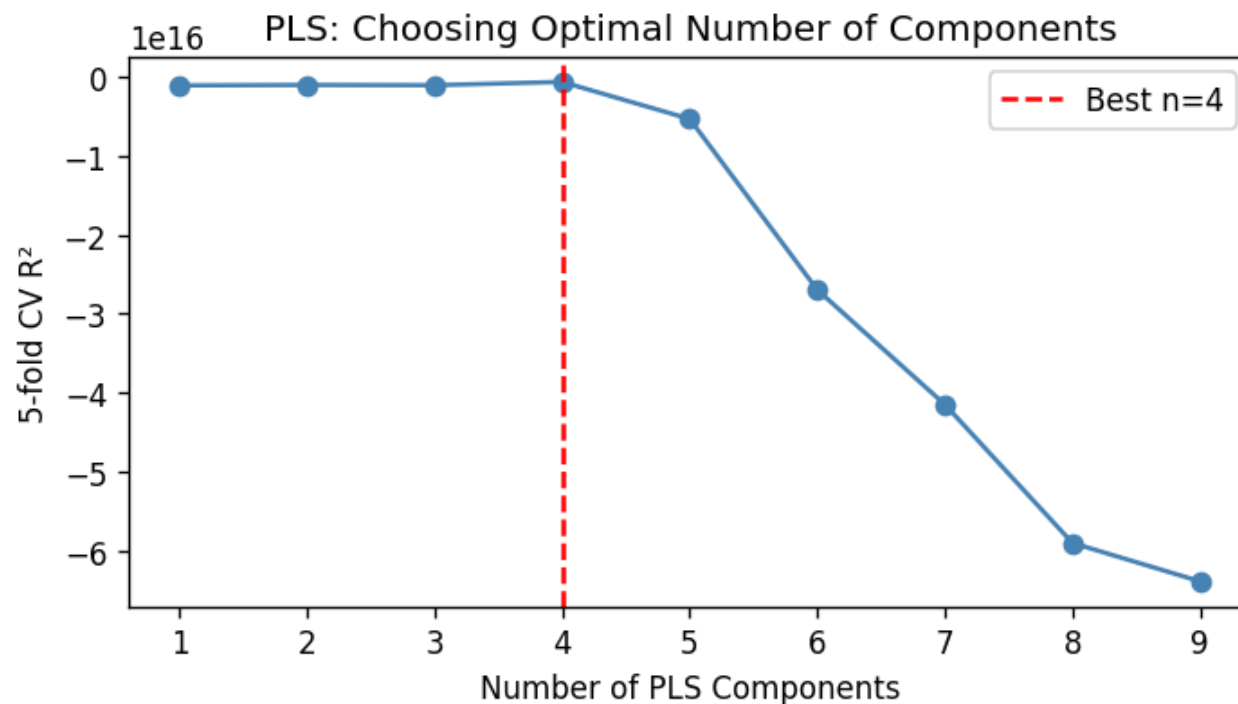
Key parameter: **n_components** — how many latent variables to use.

```
In [25]: # — 2a: PLS – find optimal number of components —————  
  
cv = KFold(n_splits=5, shuffle=True, random_state=42)  
n_comp_range = range(1, min(10, len(struct_feats)))  
  
cv_r2 = []  
for n in n_comp_range:  
    pls = PLSRegression(n_components=n)  
    cvsc = cross_val_score(pls, X_struct, y_IS, cv=cv, scoring='r2')  
    cv_r2.append(cvsc.mean())  
  
best_n = n_comp_range[int(np.argmax(cv_r2))]
```

```

fig, ax = plt.subplots(figsize=(6, 3.5))
ax.plot(list(n_comp_range), cv_r2, 'o-', color='steelblue')
ax.axvline(best_n, color='red', linestyle='--', label=f'Best n={best_n}')
ax.set_xlabel('Number of PLS Components')
ax.set_ylabel('5-fold CV R2')
ax.set_title('PLS: Choosing Optimal Number of Components')
ax.legend()
plt.tight_layout()
plt.show()
print(f'Best n_components = {best_n}, CV R2 = {cv_r2[best_n-1]:.3f}')

```



Best n_components = 4, CV R² = -645226083235968.250

```

In [26]: # — 2b: Fit PLS with best n and evaluate —————

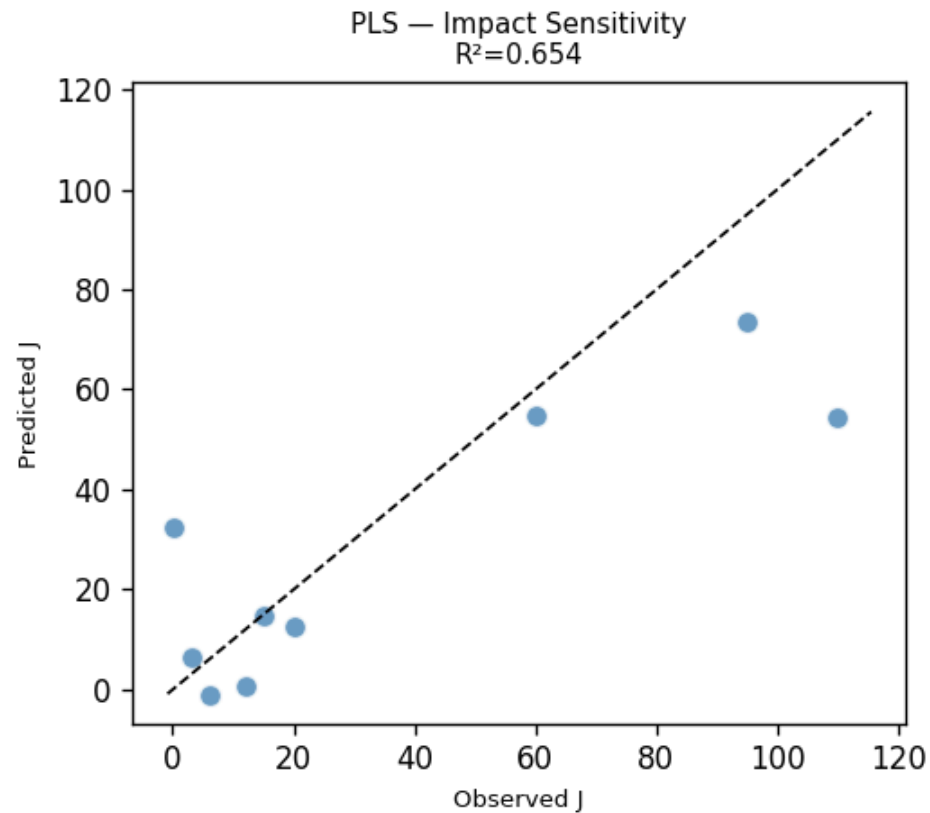
pls_best = PLSRegression(n_components=best_n)
res_pls, pred_pls = evaluate('PLS', pls_best, Xs_tr, Xs_te, ys_tr, ys_te)

print('PLS Results:')
for k, v in res_pls.items(): print(f' {k}: {v}')

fig, ax = plt.subplots(figsize=(4.5, 4))
obs_vs_pred(ax, ys_te, pred_pls, 'PLS - Impact Sensitivity', 'J')
plt.tight_layout(); plt.show()

```

PLS Results:
Model: PLS
R²_{train}: 0.754
R²_{test}: 0.654
MAE_{test}: 15.97
RMSE_{test}: 23.25



Block 3 · Support Vector Machine (SVM)

SVM finds the best-fitting surface in descriptor space.

The **kernel trick** lets it capture non-linear relationships without explicitly computing them.

Kernel	Relationship captured
linear	Straight-line only
rbf (Gaussian)	Smooth curved surfaces — most common

Kernel	Relationship captured
poly	Polynomial curves

Key parameters: **C** (how much to penalise errors) and **gamma** (kernel width).

In [29]: # — 3a: Compare SVM kernels —————

```
svm_results = []
for kernel in ['linear', 'rbf', 'poly']:
    res, _ = evaluate(f'SVM ({kernel})',
                     SVR(kernel=kernel, C=10, gamma='scale'),
                     X_tr, X_te, y_tr, y_te)
    svm_results.append(res)

df_svm = pd.DataFrame(svm_results)
display(df_svm)
```

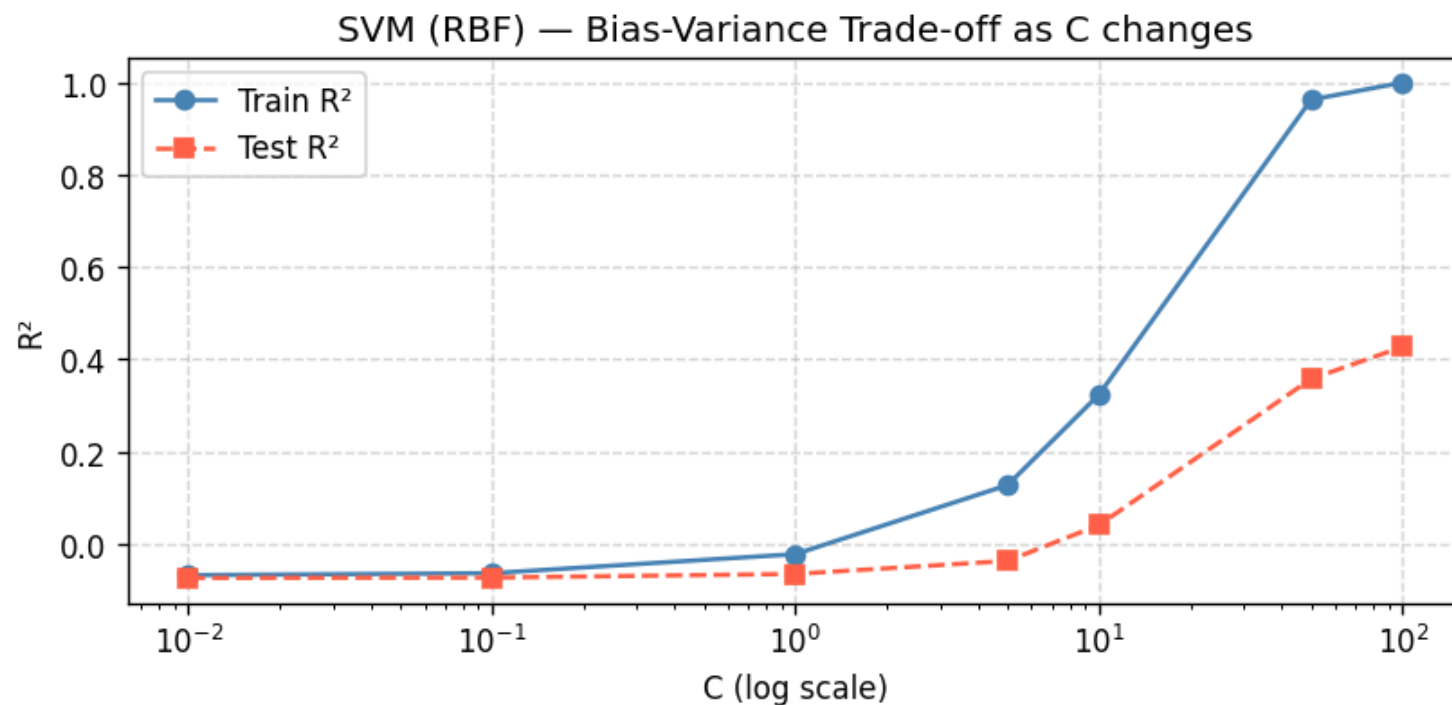
	Model	R2_train	R2_test	MAE_test	RMSE_test
0	SVM (linear)	1.000	0.535	17.22	26.97
1	SVM (rbf)	0.323	0.040	27.48	38.74
2	SVM (poly)	0.382	0.011	30.77	39.32

In [30]: # — 3b: Effect of C (regularisation) on SVM performance —————

```
C_vals = [0.01, 0.1, 1, 5, 10, 50, 100]
train_r2s, test_r2s = [], []

for C in C_vals:
    m = SVR(kernel='rbf', C=C, gamma='scale')
    m.fit(X_tr, y_tr)
    train_r2s.append(r2_score(y_tr, m.predict(X_tr)))
    test_r2s.append(r2_score(y_te, m.predict(X_te)))

fig, ax = plt.subplots(figsize=(7, 3.5))
ax.semilogx(C_vals, train_r2s, 'o-', label='Train R²', color='steelblue')
ax.semilogx(C_vals, test_r2s, 's--', label='Test R²', color='tomato')
ax.set_xlabel('C (log scale)'); ax.set_ylabel('R²')
ax.set_title('SVM (RBF) - Bias-Variance Trade-off as C changes')
ax.legend(); ax.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout(); plt.show()
print('Low C = underfitting (high bias). High C = overfitting (high variance).')
```



Low C = underfitting (high bias). High C = overfitting (high variance).

Block 4 · Random Forest

Random Forest builds **many decision trees** on random data subsets, then averages predictions.

It is robust, handles non-linearity automatically, and provides **feature importance** scores.

It is usually the **best starting model** for energetic materials QSPR.

```
In [31]: # — 4a: Fit Random Forest and show results —————

rf = RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1)
res_rf, pred_rf = evaluate('Random Forest', rf, X_tr, X_te, y_tr, y_te)

print('Random Forest Results:')
for k, v in res_rf.items(): print(f' {k}: {v}')

fig, axes = plt.subplots(1, 2, figsize=(11, 4))

# Observed vs Predicted
obs_vs_pred(axes[0], y_te, pred_rf, 'RF - Impact Sensitivity', 'J')
```

```
# Residuals
residuals = y_te - pred_rf
axes[1].scatter(pred_rf, residuals, color='steelblue', s=55, alpha=0.8, edgecolors='white')
axes[1].axhline(0, color='black', lw=1, linestyle='--')
axes[1].set_xlabel('Predicted IS (J)'); axes[1].set_ylabel('Residual (obs - pred)')
axes[1].set_title('Residual Plot — look for random scatter around zero')

plt.tight_layout(); plt.show()
```

Random Forest Results:

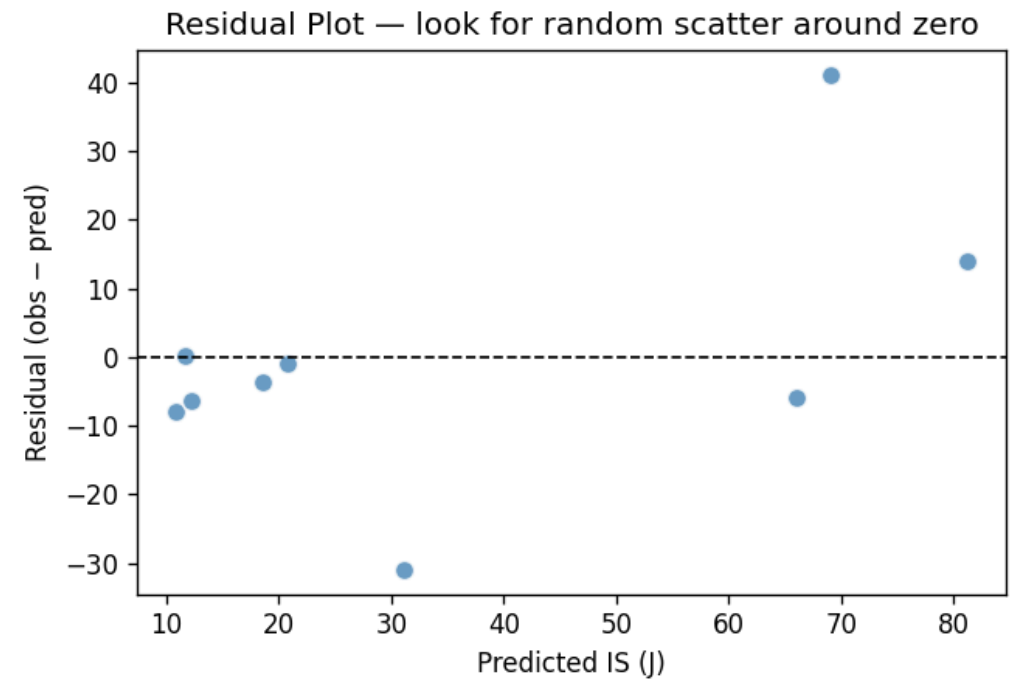
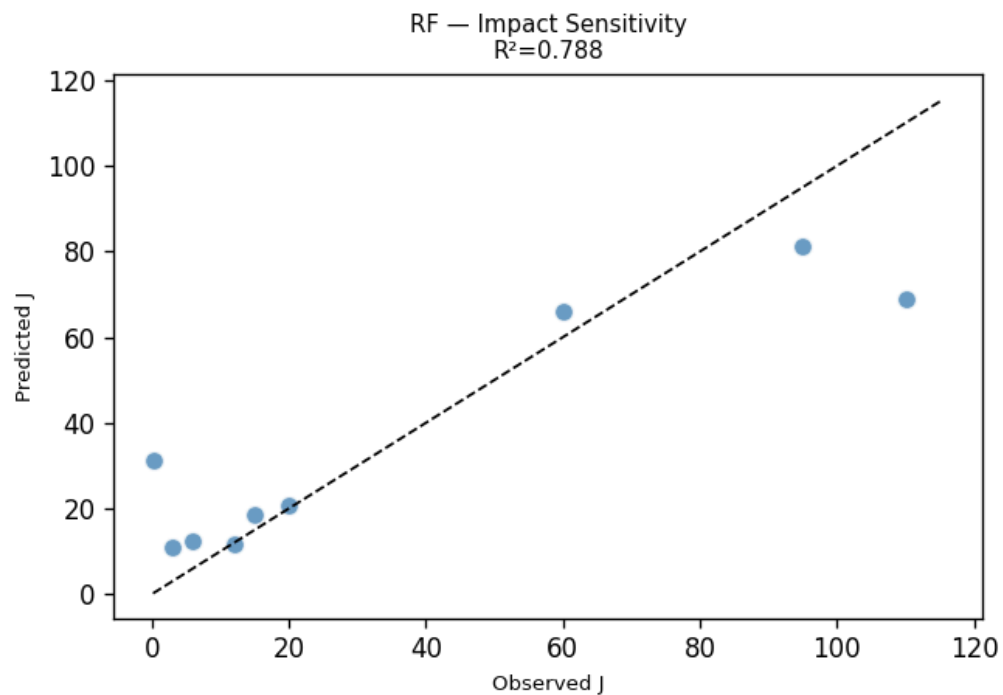
Model: Random Forest

R2_train: 0.913

R2_test: 0.788

MAE_test: 12.31

RMSE_test: 18.21



In [32]: # — 4b: Feature importance — which descriptors drive predictions?

```
importances = rf.feature_importances_
feat_imp = pd.DataFrame({'Feature': feat_names_final,
                        'Importance': importances})\
    .sort_values('Importance', ascending=False).head(20)
```

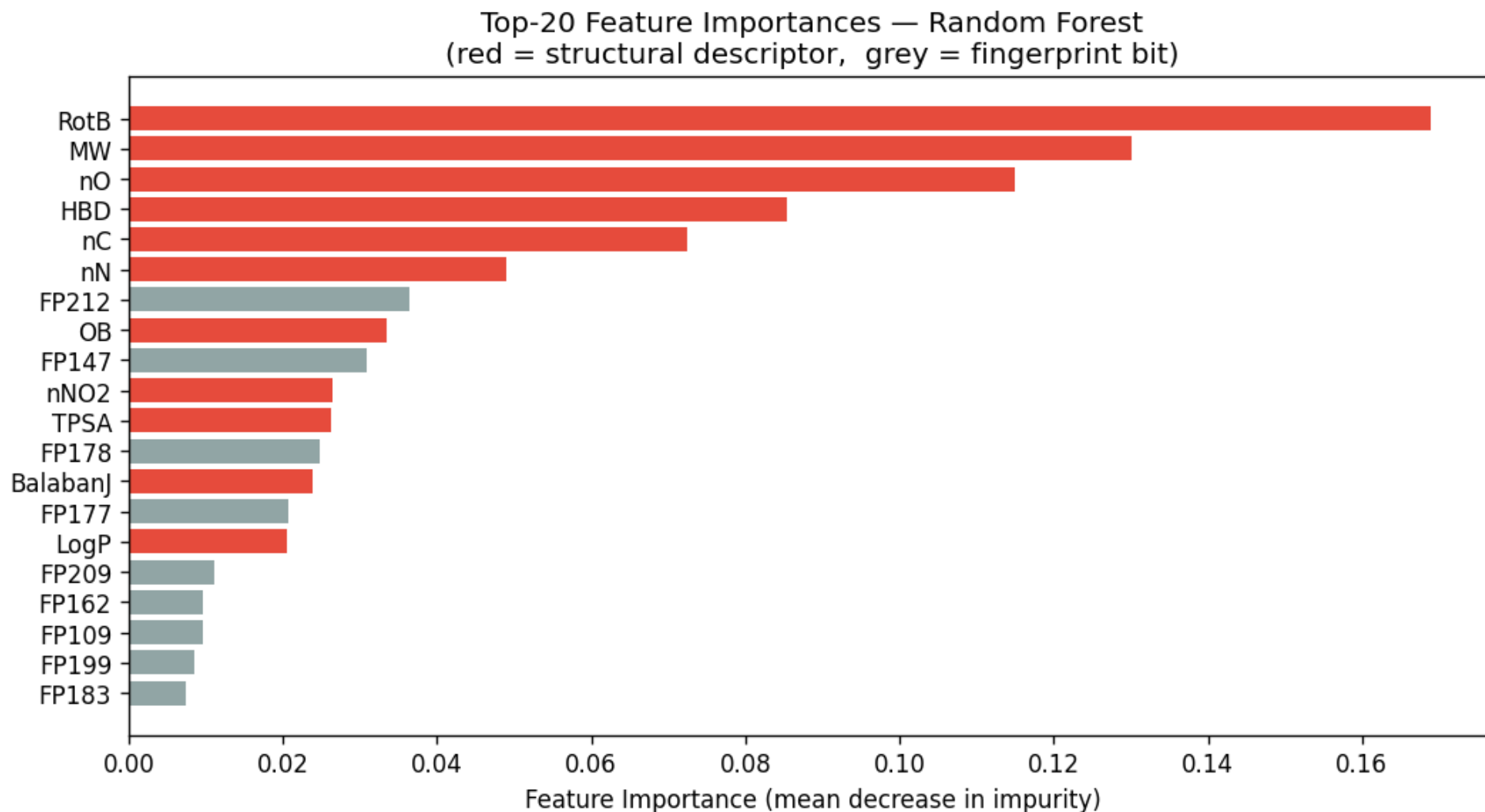
```
fig, ax = plt.subplots(figsize=(9, 5))
colors = ['#e74c3c' if not f.startswith('FP') else '#95a5a6'
```

```

for f in feat_imp['Feature']
ax.barh(feat_imp['Feature'][:-1], feat_imp['Importance'][:-1], color=colors[:-1])
ax.set_xlabel('Feature Importance (mean decrease in impurity)')
ax.set_title('Top-20 Feature Importances — Random Forest\n'
            '(red = structural descriptor, grey = fingerprint bit)')
plt.tight_layout(); plt.show()

# Print top structural descriptors
top_struct = feat_imp[~feat_imp['Feature'].str.startswith('FP')].head(5)
print('Top structural descriptors:')
print(top_struct.to_string(index=False))

```



Top structural descriptors:

Feature	Importance
RotB	0.168965
MW	0.130163
nO	0.114926
HBD	0.085402
nC	0.072459

```
In [33]: # — 4c: Effect of n_estimators on performance —————
```

```
n_tree_vals = [5, 10, 25, 50, 100, 200, 500]
```

```
tr_r2s, te_r2s = [], []
```

```
for n in n_tree_vals:
```

```
    m = RandomForestRegressor(n_estimators=n, random_state=42, n_jobs=-1)
```

```
    m.fit(X_tr, y_tr)
```

```
    tr_r2s.append(r2_score(y_tr, m.predict(X_tr)))
```

```
    te_r2s.append(r2_score(y_te, m.predict(X_te)))
```

```
fig, ax = plt.subplots(figsize=(7, 3.5))
```

```
ax.plot(n_tree_vals, tr_r2s, 'o-', label='Train R2', color='steelblue')
```

```
ax.plot(n_tree_vals, te_r2s, 's--', label='Test R2', color='tomato')
```

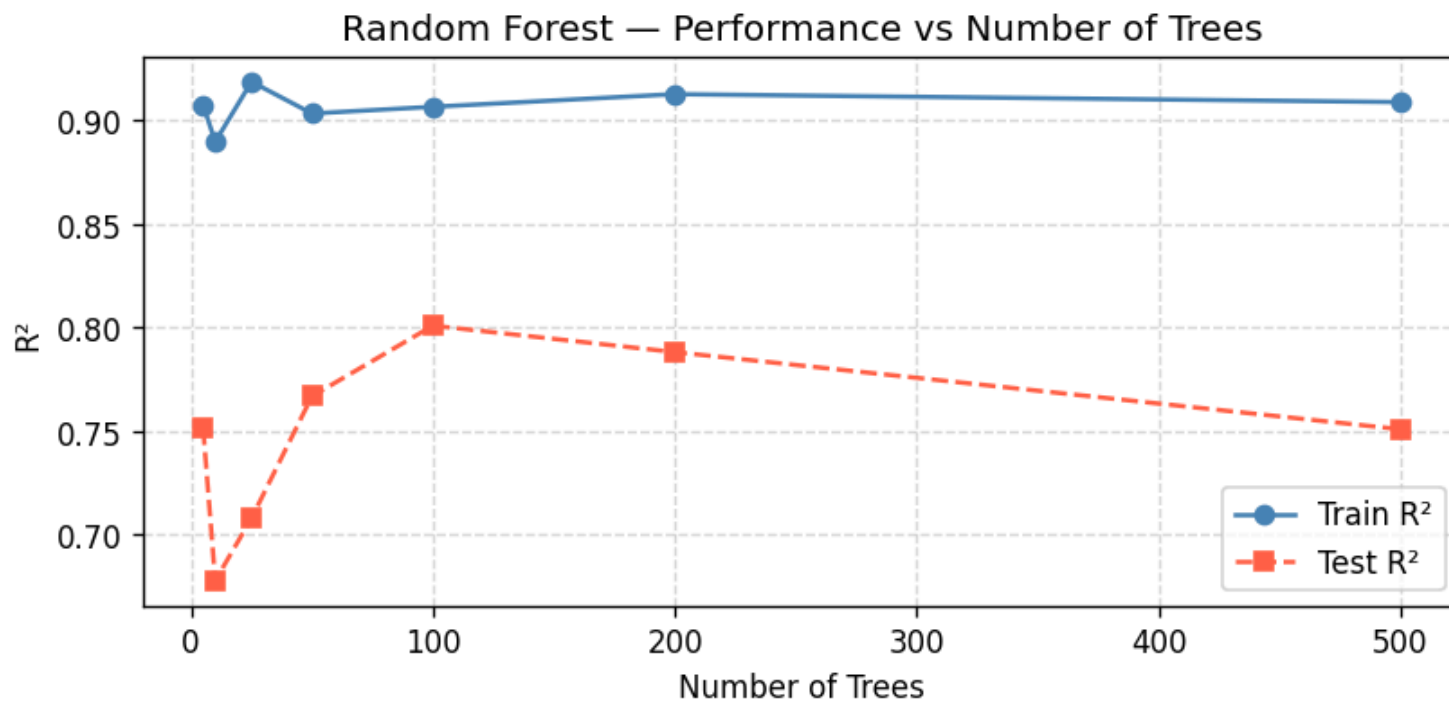
```
ax.set_xlabel('Number of Trees'); ax.set_ylabel('R2')
```

```
ax.set_title('Random Forest — Performance vs Number of Trees')
```

```
ax.legend(); ax.grid(True, linestyle='--', alpha=0.5)
```

```
plt.tight_layout(); plt.show()
```

```
print('Performance stabilises after ~100 trees. Adding more gives diminishing returns.')
```



Performance stabilises after ~100 trees. Adding more gives diminishing returns.

Block 5 · Gradient Boosting (XGBoost / sklearn)

Gradient boosting builds trees **sequentially** — each tree corrects errors from the previous. It typically achieves the **highest accuracy** on tabular datasets but needs careful tuning.

Parameter	What it controls
<code>n_estimators</code>	Number of boosting rounds
<code>learning_rate</code>	How much each tree contributes (smaller = more robust, needs more trees)
<code>max_depth</code>	Depth of each tree (deeper = more complex, more risk of overfit)
<code>subsample</code>	Fraction of rows used per tree (< 1 reduces overfit)

In [35]: `# — 5a: Gradient Boosting — sklearn vs XGBoost —`

```
gb_sklearn = GradientBoostingRegressor(
    n_estimators=200, learning_rate=0.05, max_depth=3,
    subsample=0.8, random_state=42)
```

```

res_gb, pred_gb = evaluate('GradBoost (sklearn)', gb_sklearn, X_tr, X_te, y_tr, y_te)

results_boost = [res_gb]

if XGB_OK:
    xgb_model = XGBRegressor(
        n_estimators=200, learning_rate=0.05, max_depth=3,
        subsample=0.8, colsample_bytree=0.8,
        random_state=42, verbosity=0)
    res_xgb, pred_xgb = evaluate('XGBoost', xgb_model, X_tr, X_te, y_tr, y_te)
    results_boost.append(res_xgb)

display(pd.DataFrame(results_boost))

```

	Model	R2_train	R2_test	MAE_test	RMSE_test
0	GradBoost (sklearn)	1.0	0.734	13.01	20.42
1	XGBoost	1.0	0.876	10.24	13.94

In [36]: *# — 5b: Learning curve – training loss vs boosting rounds —*

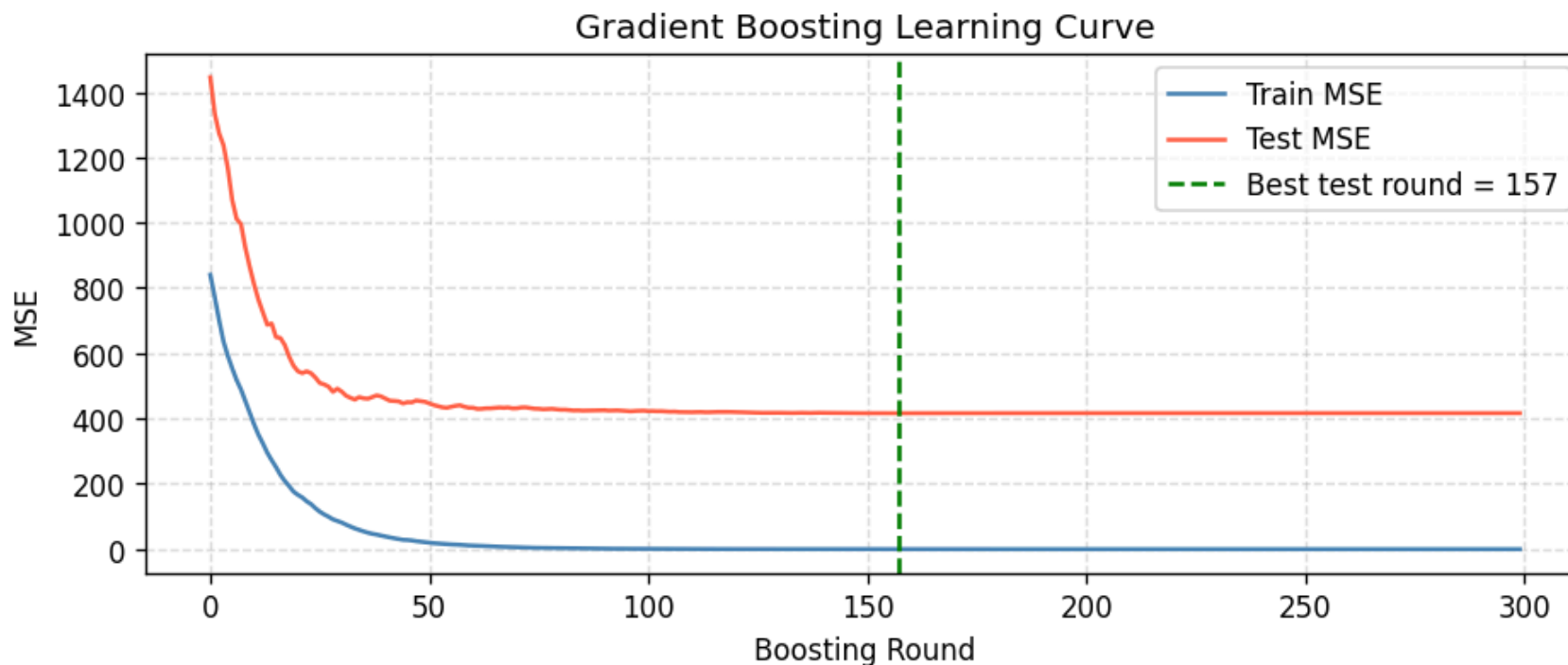
```

# Staged predictions let us watch training and test loss at each round
gb_trace = GradientBoostingRegressor(
    n_estimators=300, learning_rate=0.05, max_depth=3,
    subsample=0.8, random_state=42)
gb_trace.fit(X_tr, y_tr)

train_err = [mean_squared_error(y_tr, pred) for pred in gb_trace.staged_predict(X_tr)]
test_err = [mean_squared_error(y_te, pred) for pred in gb_trace.staged_predict(X_te)]

fig, ax = plt.subplots(figsize=(8, 3.5))
ax.plot(train_err, label='Train MSE', color='steelblue')
ax.plot(test_err, label='Test MSE', color='tomato')
best_round = int(np.argmin(test_err))
ax.axvline(best_round, color='green', linestyle='--',
            label=f'Best test round = {best_round}')
ax.set_xlabel('Boosting Round'); ax.set_ylabel('MSE')
ax.set_title('Gradient Boosting Learning Curve')
ax.legend(); ax.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout(); plt.show()
print(f'Test MSE bottoms out at round {best_round} – set n_estimators to this.')

```



Test MSE bottoms out at round 157 – set `n_estimators` to this.

Block 6 · Cross-Validation — Reliable Performance Estimation

A single train/test split gives one number that depends on which compounds land in the test set by chance.

K-fold cross-validation repeats the split K times and averages the result — much more reliable.

Rule: Always report cross-validated performance, not just one test-set R^2 .

```
In [44]: # — 6a: 5-fold and 10-fold CV for all models —————

models_cv = [
    ('Ridge', Ridge(alpha=1.0)),
    ('PLS', PLSRegression(n_components=best_n)),
    ('SVM (RBF)', SVR(kernel='rbf', C=10, gamma='scale')),
    ('Random Forest', RandomForestRegressor(n_estimators=200, random_state=42)),
    ('GradBoost', GradientBoostingRegressor(n_estimators=200, learning_rate=0.05,
                                             max_depth=3, random_state=42)),
]

cv_rows = []
```

```

for name, model in models_cv:
    cv5 = cross_val_score(model, X_scaled, y_IS, cv=5, scoring='r2')
    cv10 = cross_val_score(model, X_scaled, y_IS, cv=10, scoring='r2')
    cv_rows.append({
        'Model': name,
        '5-fold mean': round(cv5.mean(), 3),
        '5-fold std': round(cv5.std(), 3),
        '10-fold mean': round(cv10.mean(), 3),
        '10-fold std': round(cv10.std(), 3),
    })

df_cv = pd.DataFrame(cv_rows)
display(df_cv)

```

GradBoost

	Model	5-fold mean	5-fold std	10-fold mean	10-fold std
0	Ridge	NaN	NaN	NaN	NaN
1	PLS	-2.652502e+14	5.305005e+14	-2.439121e+16	7.317363e+16
2	SVM (RBF)	-2.660000e-01	5.680000e-01	-3.280100e+01	8.330100e+01
3	Random Forest	2.340000e-01	7.410000e-01	-1.585100e+01	3.746200e+01
4	GradBoost	2.680000e-01	7.800000e-01	-1.213800e+01	2.508100e+01

In [48]: *# — 6b: Visualise CV fold distribution —————*

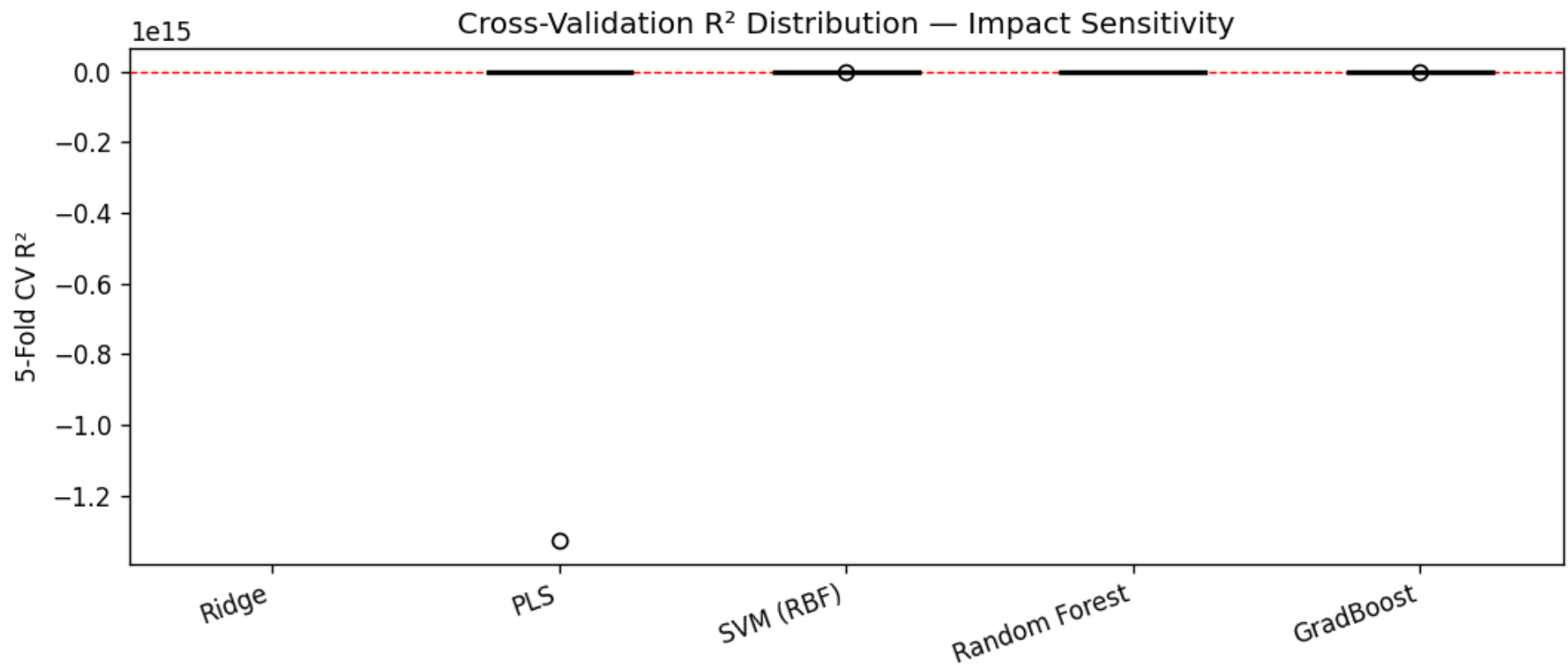
```

fig, ax = plt.subplots(figsize=(9, 4))
model_names = [r[0] for r in models_cv]

all_cv_scores = []
for name, model in models_cv:
    cv5 = cross_val_score(model, X_scaled, y_IS, cv=5, scoring='r2')
    all_cv_scores.append(cv5)

bp = ax.boxplot(all_cv_scores, labels=model_names, patch_artist=True,
                medianprops=dict(color='black', lw=2))
colors_bp = ['#AED6F1', '#A9DFBF', '#FAD7A0', '#F1948A', '#D7BDE2']
for patch, col in zip(bp['boxes'], colors_bp):
    patch.set_facecolor(col)
ax.set_ylabel('5-Fold CV R2')
ax.set_title('Cross-Validation R2 Distribution – Impact Sensitivity')
ax.axhline(0, color='red', linestyle='--', linewidth=0.8)
ax.set_xticklabels(model_names, rotation=20, ha='right')
plt.tight_layout(); plt.show()
print('Narrow box = consistent across folds. Wide box = model is sensitive to data split.')

```



Narrow box = consistent across folds. Wide box = model is sensitive to data split.

Block 7 · Hyperparameter Tuning — Grid Search and Random Search

Most models have **hyperparameters** (settings you choose before training).

Tuning finds the best combination automatically.

Method	How it works	When to use
Grid Search	Tries every combination in a predefined grid	Small grids (<100 combos)
Random Search	Randomly samples combinations	Large grids — often finds good settings faster

In [54]: # — 7a: Grid search — SVM —————

```
svm_grid = {
    'C': [0.1, 1, 10, 50],
    'gamma': ['scale', 0.01, 0.001],
```

```

    'kernel':['rbf', 'linear'],
}

gs = GridSearchCV(SVR(), svm_grid, cv=5, scoring='r2', n_jobs=-1, verbose=0)
gs.fit(X_scaled, y_IS)

print('Grid Search - SVM')
print(f' Best params: {gs.best_params_}')
print(f' Best CV R²: {gs.best_score_:.3f}')

# Show top 5 combinations
cv_results = pd.DataFrame(gs.cv_results_) \
    .sort_values('mean_test_score', ascending=False) \
    [['param_C', 'param_gamma', 'param_kernel', 'mean_test_score', 'std_test_score']] \
    .head(5)
cv_results.columns = ['C', 'gamma', 'kernel', 'mean_R2', 'std_R2']
display(cv_results)

```

Grid Search - SVM

Best params: {'C': 1, 'gamma': 'scale', 'kernel': 'linear'}

Best CV R²: -0.001

	C	gamma	kernel	mean_R2	std_R2
7	1	scale	linear	-0.000898	0.899174
9	1	0.01	linear	-0.000898	0.899174
11	1	0.001	linear	-0.000898	0.899174
3	0.1	0.01	linear	-0.044208	0.600026
5	0.1	0.001	linear	-0.044208	0.600026

In [55]: # — 7b: Random search - Random Forest —————

```

from scipy.stats import randint, uniform

rf_param_dist = {
    'n_estimators': randint(50, 400),
    'max_depth': [None, 3, 5, 8, 10],
    'min_samples_split': randint(2, 10),
    'min_samples_leaf': randint(1, 5),
    'max_features': ['sqrt', 'log2', 0.5],
}

rs = RandomizedSearchCV(
    RandomForestRegressor(random_state=42, n_jobs=-1),
    rf_param_dist, n_iter=30, cv=5,
    scoring='r2', random_state=42, n_jobs=-1, verbose=0)
rs.fit(X_scaled, y_IS)

```

```

print('Random Search - Random Forest')
print(f' Best params: {rs.best_params_}')
print(f' Best CV R²: {rs.best_score_:.3f}')

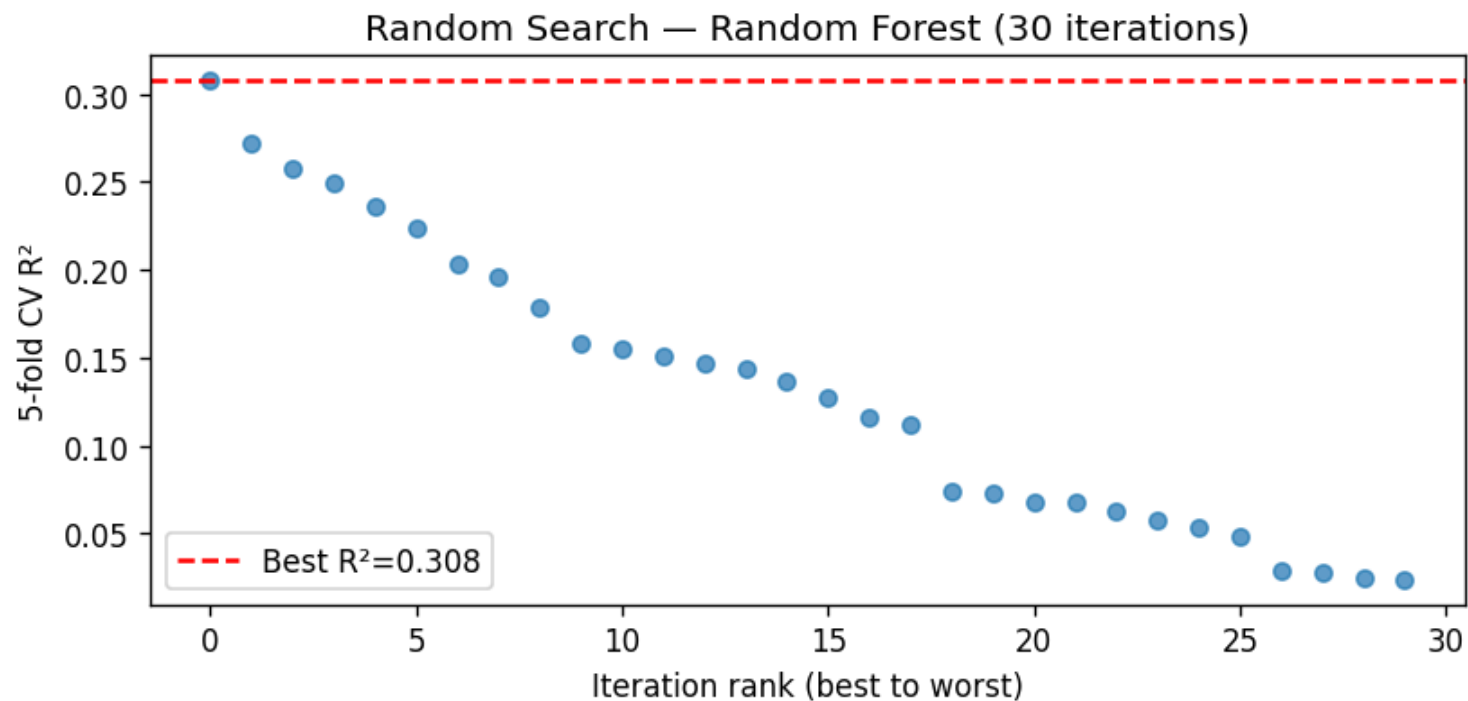
# Plot improvement over iterations
rs_df = pd.DataFrame(rs.cv_results_).sort_values('rank_test_score')
fig, ax = plt.subplots(figsize=(7, 3.5))
ax.scatter(range(len(rs_df)), rs_df['mean_test_score'], s=30, alpha=0.7)
ax.axhline(rs.best_score_, color='red', linestyle='--',
           label=f'Best R²={rs.best_score_:.3f}')
ax.set_xlabel('Iteration rank (best to worst)')
ax.set_ylabel('5-fold CV R²')
ax.set_title('Random Search - Random Forest (30 iterations)')
ax.legend(); plt.tight_layout(); plt.show()

```

Random Search - Random Forest

Best params: {'max_depth': 10, 'max_features': 0.5, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 394}

Best CV R²: 0.308



Compare all models side-by-side on the same dataset using the same split.
Use CV R^2 as the primary metric — not training R^2 .

```
In [61]: # — 8a: Train all models and collect metrics —————

best_svm = gs.best_estimator_
best_rf  = rs.best_estimator_

final_models = [
    # Change solver to 'svd' or 'sag' to avoid the sym_pos error
    ('Ridge',      Ridge(alpha=1.0, solver='svd')),
    ('PLS',        PLSRegression(n_components=best_n)),
    ('SVM (tuned)', best_svm),
    ('RF (tuned)',  best_rf),
    ('GradBoost',  GradientBoostingRegressor(n_estimators=200, learning_rate=0.05,
                                              max_depth=3, random_state=42)),
]

dashboard_rows = []
pred_all = {}
for name, model in final_models:
    cv5 = cross_val_score(model, X_scaled, y_IS, cv=5, scoring='r2')
    res, pred = evaluate(name, model, X_tr, X_te, y_tr, y_te)
    res['CV5_mean'] = round(cv5.mean(), 3)
    res['CV5_std']  = round(cv5.std(), 3)
    dashboard_rows.append(res)
    pred_all[name] = pred

df_dash = pd.DataFrame(dashboard_rows)
display(df_dash)
```

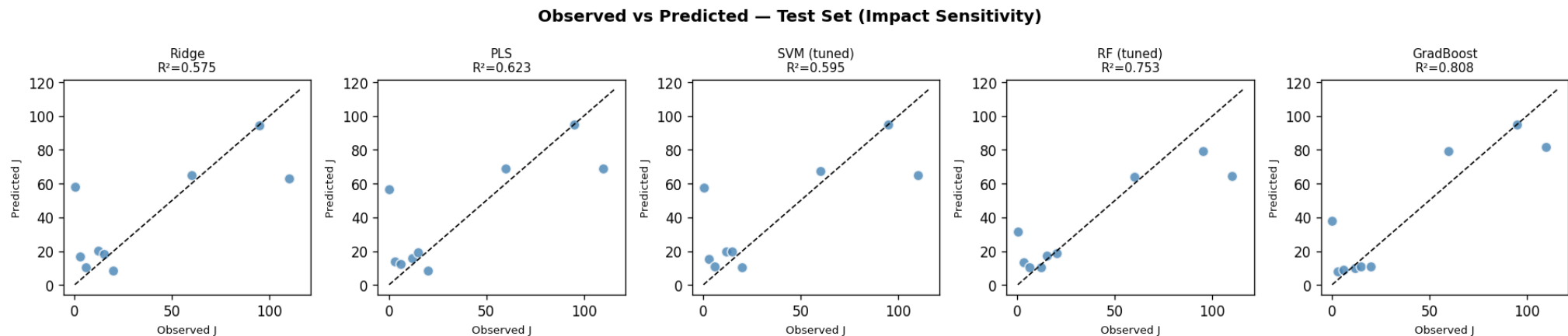
	Model	R2_train	R2_test	MAE_test	RMSE_test	CV5_mean	CV5_std
0	Ridge	0.999	0.575	16.82	25.79	-4.800000e-02	9.490000e-01
1	PLS	0.996	0.623	16.00	24.27	-2.652502e+14	5.305005e+14
2	SVM (tuned)	0.996	0.595	16.56	25.18	-1.000000e-03	8.990000e-01
3	RF (tuned)	0.915	0.753	12.95	19.66	3.080000e-01	6.790000e-01
4	GradBoost	1.000	0.808	11.96	17.35	2.680000e-01	7.800000e-01

```
In [62]: # — 8b: Obs vs Pred grid for all models —————
```

```
fig, axes = plt.subplots(1, len(final_models), figsize=(16, 3.5))

for ax, (name, _) in zip(axes, final_models):
    obs_vs_pred(ax, y_te, pred_all[name], name, 'J')

plt.suptitle('Observed vs Predicted – Test Set (Impact Sensitivity)',
             fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()
```



```
In [63]: # — 8c: Radar comparison chart —————

import matplotlib.patches as mpatches

metrics      = ['R2_train', 'R2_test', 'CV5_mean']
inv_metrics = ['MAE_test', 'RMSE_test'] # Lower=better → invert for radar

# Normalise 0-1 for radar
radar_df = df_dash.set_index('Model')[metrics + inv_metrics].copy()
for col in inv_metrics: # invert so bigger = better
    radar_df[col] = 1 - (radar_df[col] - radar_df[col].min()) / \
        (radar_df[col].max() - radar_df[col].min() + 1e-9)
for col in metrics:
    radar_df[col] = (radar_df[col] - radar_df[col].min()) / \
        (radar_df[col].max() - radar_df[col].min() + 1e-9)

all_labels = metrics + [f'1/{m}' for m in inv_metrics]
angles = np.linspace(0, 2*np.pi, len(all_labels), endpoint=False).tolist()
angles += angles[:1]

fig, ax = plt.subplots(figsize=(6, 6), subplot_kw=dict(polar=True))
colors_r = plt.cm.Set2.colors

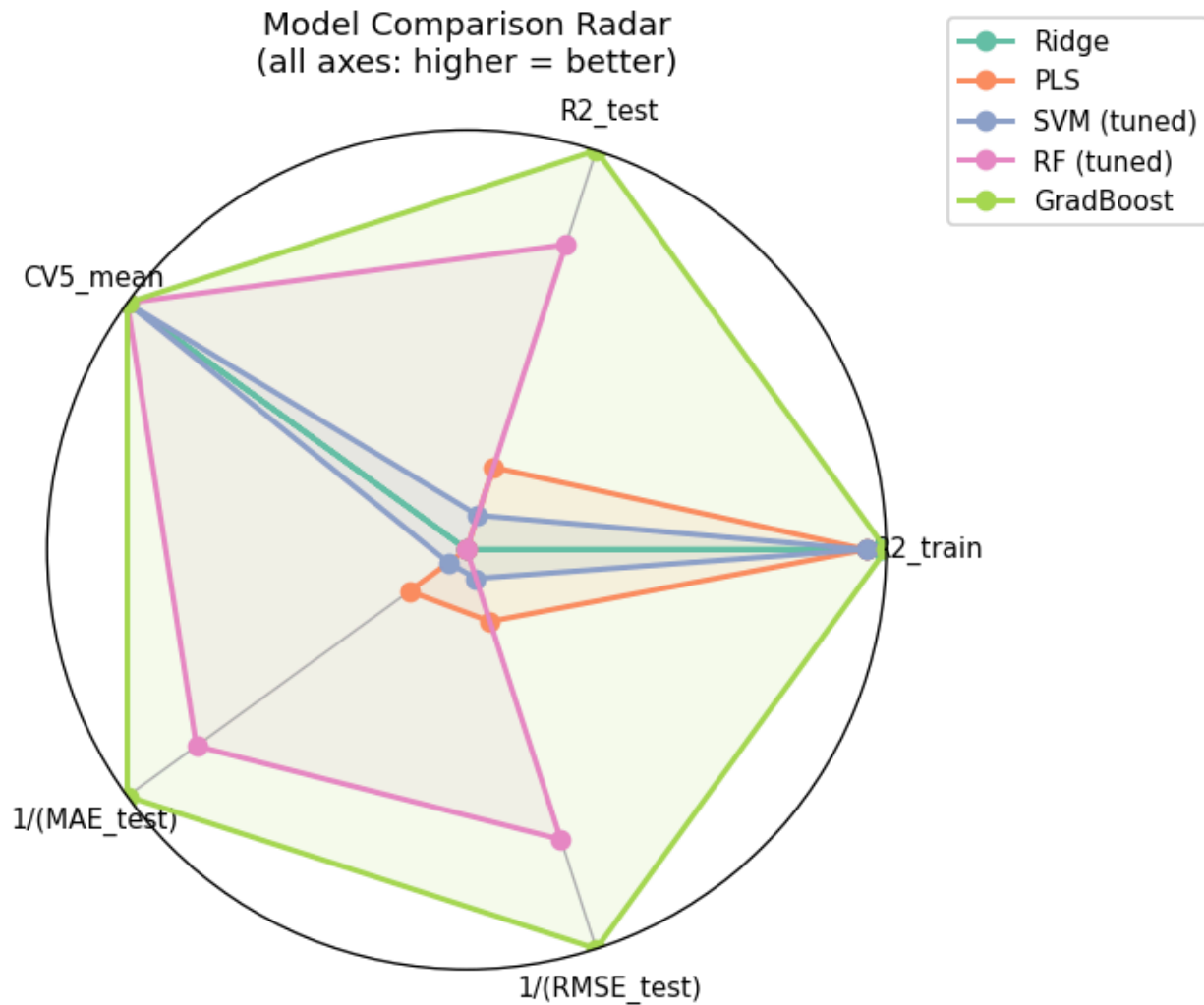
for idx, (model_name, row) in enumerate(radar_df.iterrows()):
    vals = row.tolist() + row.tolist()[:1]
```

```

ax.plot(angles, vals, 'o-', lw=1.8, color=colors_r[idx], label=model_name)
ax.fill(angles, vals, alpha=0.1, color=colors_r[idx])

ax.set_thetagrids(np.degrees(angles[:-1]), all_labels, fontsize=9)
ax.set_ylim(0, 1); ax.set_yticks([])
ax.set_title('Model Comparison Radar\n(all axes: higher = better)', fontsize=11, pad=20)
ax.legend(loc='upper right', bbox_to_anchor=(1.4, 1.15), fontsize=9)
plt.tight_layout(); plt.show()

```



Chapter 4 covers five prediction targets.

Here we train the best model (Random Forest) separately for each property and plot all observed-vs-predicted results together.

```
In [65]: # — 9a: Train RF for each property —————

targets = {
    'Impact Sensitivity (J)': y_IS,
    'Crystal Density (g/cc)': y_rho,
    'Detonation Velocity (m/s)': y_Dv,
    'Decomp. Temperature (°C)': y_Td,
    'Heat of Formation (kJ/mol)': y_Hf,
}

prop_results = []
fig, axes = plt.subplots(2, 3, figsize=(14, 8))
axes = axes.flatten()

for idx, (label, y_prop) in enumerate(targets.items()):
    Xp_tr, Xp_te, yp_tr, yp_te = train_test_split(
        X_scaled, y_prop, test_size=0.30, random_state=42)

    rf_prop = RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1)
    rf_prop.fit(Xp_tr, yp_tr)
    yp_pred = rf_prop.predict(Xp_te)

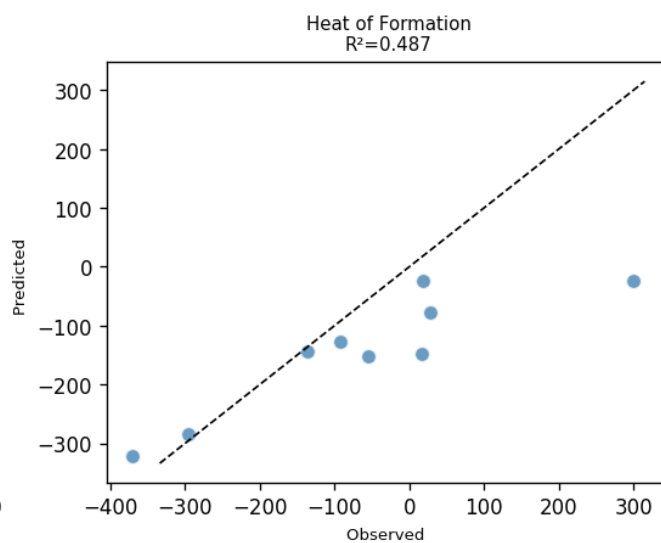
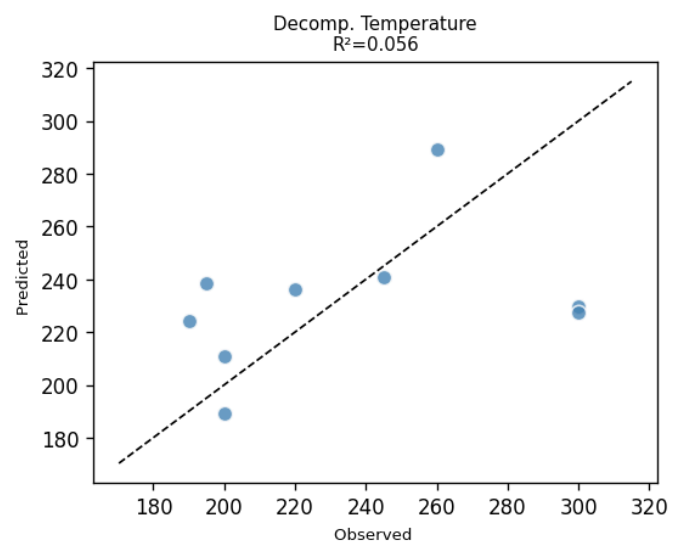
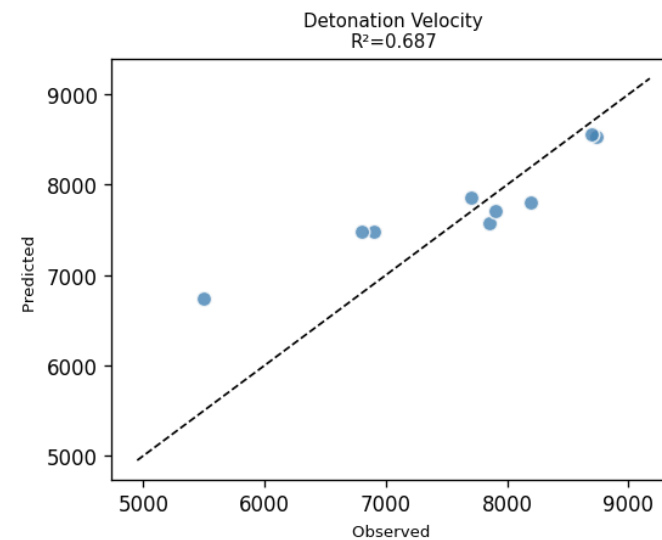
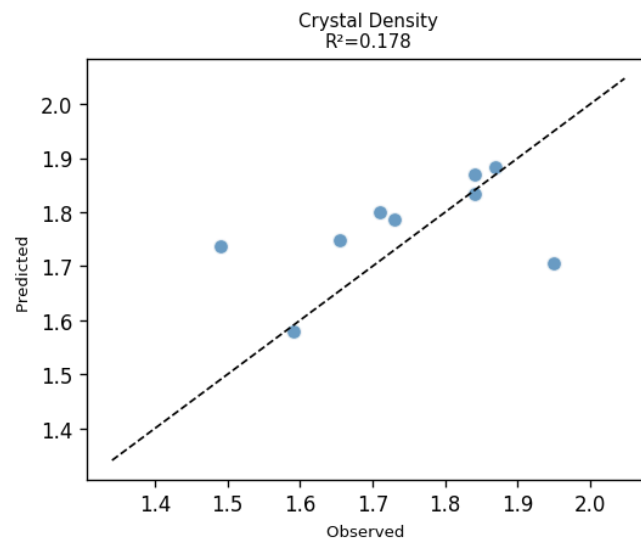
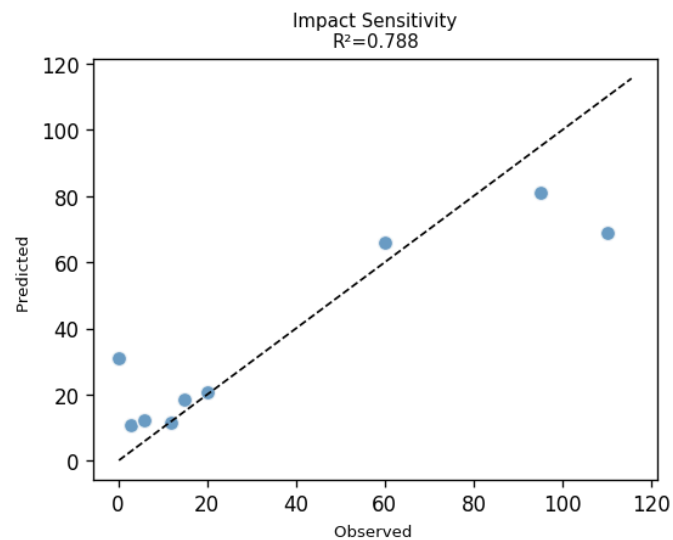
    r2 = r2_score(yp_te, yp_pred)
    mae = mean_absolute_error(yp_te, yp_pred)
    prop_results.append({'Property': label,
                        'R2_test': round(r2, 3),
                        'MAE_test': round(mae, 2)})

    ax = axes[idx]
    obs_vs_pred(ax, yp_te, yp_pred, label.split('(')[0].strip())

axes[-1].axis('off') # hide unused 6th subplot
plt.suptitle('Random Forest – All Property Predictions (Test Set)',
            fontsize=13, fontweight='bold')
plt.tight_layout(); plt.show()

display(pd.DataFrame(prop_results))
```

Random Forest — All Property Predictions (Test Set)



	Property	R2_test	MAE_test
0	Impact Sensitivity (J)	0.788	12.31
1	Crystal Density (g/cc)	0.178	0.09
2	Detonation Velocity (m/s)	0.687	429.04
3	Decomp. Temperature (°C)	0.056	32.41
4	Heat of Formation (kJ/mol)	0.487	93.03

```
In [66]: # — 9b: Kamlet-Jacobs vs ML – detonation velocity —————
# Compare empirical KJ equations with RF predictions

def kamlet_jacobs_D(N, M, Q, rho):
    phi = N * math.sqrt(M * Q)
    return round(1.01 * math.sqrt(phi) * (1 + 1.30*rho) * 1000, 0)

# KJ parameters for the 4 compounds we have data for
kj_known = {
    'TNT': (0.01987, 50.51, 1080, 1.654),
    'RDX': (0.02987, 27.67, 1510, 1.820),
    'HMX': (0.02975, 27.61, 1482, 1.905),
    'PETN': (0.02865, 28.26, 1510, 1.778),
}

comp_rows = []
for cname, (N, M, Q, rho) in kj_known.items():
    idx = list(names).index(cname)
    rf_d = RandomForestRegressor(n_estimators=200, random_state=42)
    rf_d.fit(X_scaled, y_Dv)
    rf_pred = round(rf_d.predict(X_scaled[[idx]])[0], 0)
    comp_rows.append({
        'Compound': cname,
        'Experimental': df_meta.loc[df_meta['Name']==cname, 'Dv_ms'].values[0],
        'KJ Estimate': kamlet_jacobs_D(N, M, Q, rho),
        'RF Predicted': rf_pred,
    })

df_kj_vs_rf = pd.DataFrame(comp_rows)
df_kj_vs_rf['KJ Error %'] = abs(df_kj_vs_rf['KJ Estimate'] - df_kj_vs_rf['Experimental'])\
    / df_kj_vs_rf['Experimental'] * 100
df_kj_vs_rf['RF Error %'] = abs(df_kj_vs_rf['RF Predicted'] - df_kj_vs_rf['Experimental'])\
    / df_kj_vs_rf['Experimental'] * 100
df_kj_vs_rf[['KJ Error %', 'RF Error %']] = df_kj_vs_rf[['KJ Error %', 'RF Error %']].round(1)

display(df_kj_vs_rf)
```

	Compound	Experimental	KJ Estimate	RF Predicted	KJ Error %	RF Error %
0	TNT	6900	6854.0	7056.0	0.7	2.3
1	RDX	8750	8400.0	8677.0	4.0	0.8
2	HMX	9110	8614.0	8926.0	5.4	2.0
3	PETN	8400	8136.0	8482.0	3.1	1.0

A QSPR model is only reliable for molecules **similar to its training set**.

The domain of applicability tells you when NOT to trust a prediction.

Three methods covered:

1. **Leverage (hat values)** — distance from training data centroid in descriptor space
2. **Euclidean distance** to nearest training compound
3. **Prediction interval** — how much the forest's trees disagree

```
In [67]: # — 10a: Leverage method —————
#  $h = x^T (X^T X)^{-1} x$ 
# Warning threshold:  $h^* = 3(p+1)/n$  where  $p$ =features,  $n$ =train samples

from numpy.linalg import pinv

# Use structural descriptors only for interpretability
struct_idx = [i for i, f in enumerate(feats_names_final)
               if not f.startswith('FP')]
Xs_tr_doa = X_tr[:, struct_idx]
Xs_te_doa = X_te[:, struct_idx]
Xs_all_doa = X_scaled[:, struct_idx]

H_matrix = Xs_tr_doa @ pinv(Xs_tr_doa.T @ Xs_tr_doa) @ Xs_tr_doa.T
h_train = np.diag(H_matrix)

# Leverage for test set
p = Xs_tr_doa.shape[1]
n = Xs_tr_doa.shape[0]
h_star = 3 * (p + 1) / n

h_test = np.array([
    float(x @ pinv(Xs_tr_doa.T @ Xs_tr_doa) @ x)
    for x in Xs_te_doa
])

rf_doa = RandomForestRegressor(n_estimators=200, random_state=42)
rf_doa.fit(X_tr, y_tr)
pred_doa = rf_doa.predict(X_te)
std_res = np.sqrt(mean_squared_error(y_tr, rf_doa.predict(X_tr)))
std_res = max(std_res, 1e-6)
s_test = (y_te - pred_doa) / std_res

print(f'Warning threshold  $h^* = \{h\_star:.3f\}$ ')
print(f'Test compounds with  $h > h^*$ : '
      f'[{n_te[i] for i in range(len(h_test)) if h_test[i] > h_star}]')

# Williams plot: Leverage vs standardised residual
fig, ax = plt.subplots(figsize=(7, 5))
```

```

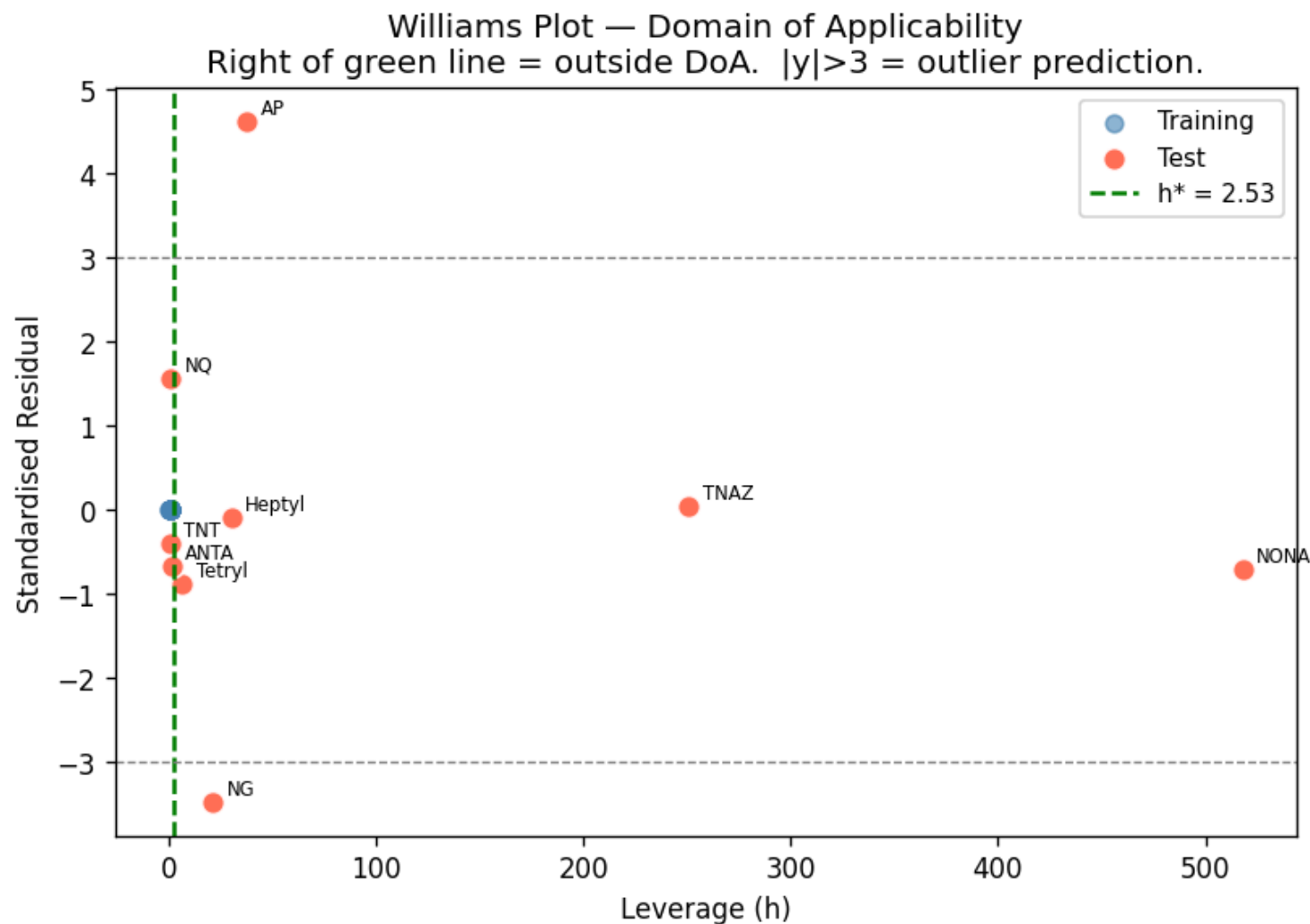
ax.scatter(h_train, np.zeros(len(h_train)), s=40, color='steelblue',
           label='Training', alpha=0.6)
ax.scatter(h_test, s_test, s=70, color='tomato',
           label='Test', alpha=0.9, edgecolors='white')
for i, name in enumerate(n_te):
    ax.annotate(name, (h_test[i], s_test[i]),
               textcoords='offset points', xytext=(5, 3), fontsize=7)

ax.axvline(h_star, color='green', linestyle='--', label=f'h* = {h_star:.2f}')
ax.axhline(3, color='grey', linestyle='--', linewidth=0.8)
ax.axhline(-3, color='grey', linestyle='--', linewidth=0.8)
ax.set_xlabel('Leverage (h)')
ax.set_ylabel('Standardised Residual')
ax.set_title('Williams Plot – Domain of Applicability\n'
             'Right of green line = outside DoA. |y|>3 = outlier prediction.')
ax.legend(fontsize=9)
plt.tight_layout(); plt.show()

```

Warning threshold $h^* = 2.526$

Test compounds with $h > h^*$: ['TNAZ', 'AP', 'Tetryl', 'NONA', 'NG', 'Heptyl']



```
In [68]: # — 10b: Euclidean distance to nearest training neighbour ———

from sklearn.metrics import pairwise_distances

dist_matrix = pairwise_distances(X_te, X_tr)
min_dist    = dist_matrix.min(axis=1) # distance to nearest training compound

# Threshold = 95th percentile of training-set internal distances
train_self_dist = pairwise_distances(X_tr, X_tr)
np.fill_diagonal(train_self_dist, np.nan)
thresh_dist = np.nanpercentile(train_self_dist, 95)

doa_df = pd.DataFrame({
    'Compound':    n_te,
```

```

'Min_dist':    min_dist.round(3),
'Observed IS': y_te,
'Predicted IS': pred_doa.round(1),
'In DoA':      min_dist < thresh_dist
})

print(f'Distance threshold (95th pct): {thresh_dist:.3f}')
display(doa_df)

```

Distance threshold (95th pct): 15.784

	Compound	Min_dist	Observed IS	Predicted IS	In DoA
0	TNAZ	11.096	12.0	11.7	True
1	AP	10.244	110.0	69.0	True
2	Tetryl	9.654	3.0	10.9	True
3	NONA	1.260	6.0	12.3	True
4	TNT	10.203	15.0	18.6	True
5	NQ	0.000	95.0	81.1	True
6	NG	1.272	0.2	31.1	True
7	Heptyl	9.894	20.0	20.8	True
8	ANTA	11.454	60.0	66.0	True

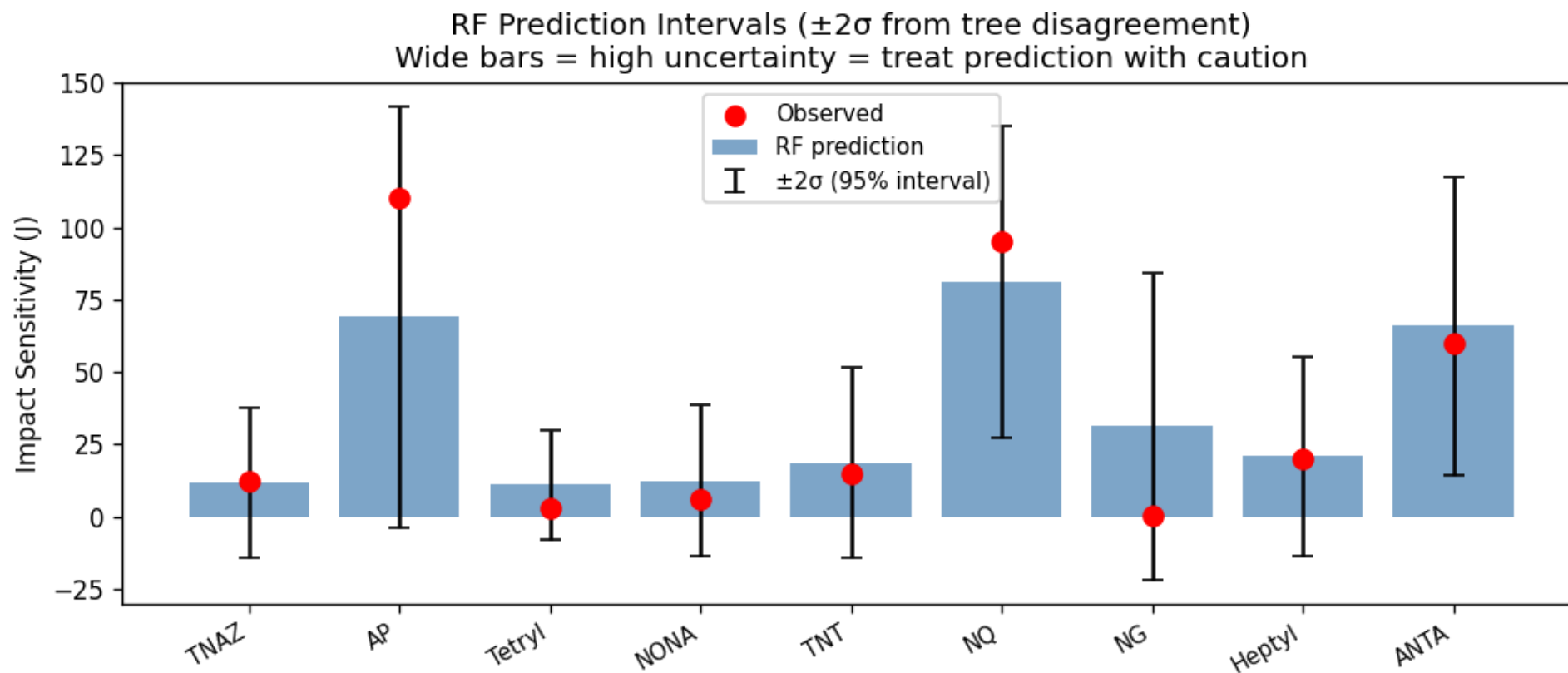
```

In [69]: # — 10c: Prediction interval from RF tree disagreement —————
# Each tree in RF gives a prediction; std across trees = confidence

tree_preds = np.array([tree.predict(X_te) for tree in rf_doa.estimators_])
pred_mean  = tree_preds.mean(axis=0)
pred_std   = tree_preds.std(axis=0)

fig, ax = plt.subplots(figsize=(9, 4))
x_pos = np.arange(len(n_te))
ax.bar(x_pos, pred_mean, color='steelblue', alpha=0.7, label='RF prediction')
ax.errorbar(x_pos, pred_mean, yerr=2*pred_std, fmt='none',
            color='black', capsize=4, label='±2σ (95% interval)')
ax.scatter(x_pos, y_te, color='red', zorder=5, s=60, label='Observed')
ax.set_xticks(x_pos)
ax.set_xticklabels(n_te, rotation=30, ha='right', fontsize=9)
ax.set_ylabel('Impact Sensitivity (J)')
ax.set_title('RF Prediction Intervals (±2σ from tree disagreement)\n'
            'Wide bars = high uncertainty = treat prediction with caution')
ax.legend(fontsize=9)
plt.tight_layout(); plt.show()

```



Summary — What You Learned in Chapter 4

Block	Concept	Key tool
0	Dataset + feature matrix setup	<code>rdkit</code> , <code>pandas</code> , <code>StandardScaler</code>
1	MLR, Ridge, Lasso — linear models + coefficients	<code>LinearRegression</code> , <code>LassoCV</code>
2	PLS — choose <code>n_components</code> by CV	<code>PLSRegression</code> , <code>cross_val_score</code>
3	SVM — kernels + C parameter effect	<code>SVR</code> , bias-variance plot
4	Random Forest — feature importance + <code>n_estimators</code>	<code>RandomForestRegressor</code>
5	Gradient Boosting + learning curve	<code>GradientBoostingRegressor</code> , <code>XGBRegressor</code>
6	K-fold cross-validation + boxplots	<code>cross_val_score</code> , <code>KFold</code>
7	Hyperparameter tuning — grid + random search	<code>GridSearchCV</code> , <code>RandomizedSearchCV</code>

Block	Concept	Key tool
8	Full model comparison dashboard + radar	All models side-by-side
9	Predict all 5 properties + KJ vs RF comparison	Multi-target RF
10	Domain of applicability — leverage, distance, intervals	Williams plot, <code>pairwise_distances</code>

Next → Chapter 5: High-throughput screening, multiscale modelling, and generative AI.

References & Tools

- Scikit-learn user guide: https://scikit-learn.org/stable/user_guide.html
- XGBoost documentation: <https://xgboost.readthedocs.io>
- Kamlet & Jacobs (1968) *J. Chem. Phys.* 48(1) — KJ detonation equations
- Tropsha, A. (2010) Best Practices for QSAR Model Development, Validation and Exploitation, *Mol. Inf.* 29(6)
- Gramatica, P. (2007) Principles of QSAR Modelling, *QSAR Comb. Sci.* 26
- Rice, B.M. and Hare, J.J. (2002) *J. Phys. Chem. A* 106(9) — sensitivity predictors
- Karthikeyan & Vyas *Practical Chemoinformatics*, Springer, 2014

In []: